Indian Institute of Technology Hyderabad

AI3403 – Multi-Agent Systems

TrafficSync

Designing an Intelligent Traffic Signal Network
Using Shared Traffic Information

Final Project Report

Group MadmaX

Abhijeet	ai24btech11001
Charan	ai24btech11022
Sumanth	ai24btech11027
Shreyansh	ai24btech11032
Tanush	ai24btech11034

Submitted to Dr. Venkatraman Ranganathan, AI3403 Multi-Agent Systems

November 2026

Abstract

Urban signalised networks are coupled systems: a queue released at one junction becomes the arrival stream of the next, and a queue that fills a short link blocks the junction upstream. Fixed-time plans cannot react to these interactions, and independently adaptive signals react only to what they can see locally. TrafficSync studies whether letting every intersection act as an autonomous agent that observes its own approaches, exchanges messages with its neighbours over an imperfect channel, and coordinates through distributed algorithms improves network performance, and under which conditions.

We implemented a deterministic vehicle-level point-queue simulator with spillback, a probe-vehicle information model and a message bus with latency, loss and outages, and on top of it a family of controllers: fixed-time Webster plans, queue-actuated and local max-pressure control, neighbour-informed max-pressure (CoMP), and consensus-cycle control with formation-control offsets (CoCycle). The controllers are extended with resilient W-MSR consensus, ADMM and gradient-tracking optimisation, distributed MPC with Signal Temporal Logic constraints, an epistemic guard and multi-agent reinforcement learning (IQL, VDN, QMIX). All comparisons are paired over common seeds (30 for the main comparison) with bootstrap confidence intervals, Wilcoxon tests and Holm correction; the controllers were also run in SUMO on six real-city scenarios.

With local detectors on a 3×3 grid, tuned actuated control beats fixed-time by 8–49%, and CoCycle reduces delay by a further 6.7% at medium and 8.1% at tidal demand. Ablations, communication-graph sweeps and message-loss experiments show that this gain is produced by CoCycle's local plan structure, not by coordination, and it did not transfer to SUMO. With 20% probe vehicles every adaptive controller is worse than fixed-time except under ramping demand. The course theory was confirmed exactly where it applies: convergence speed follows λ_2, W-MSR resists a faulty agent on a sufficiently robust graph, and a spectral epidemic threshold separates the scenarios in which congestion spreads. In six real-city scenarios the best agent design beat the city's own signal plan by 7–56%.

Keywords: multi-agent systems; traffic signal control; distributed consensus; max-pressure control; resilient consensus; distributed model predictive control; multi-agent reinforcement learning.

GitHub link for the project: `https://github.com/abhijeet-iith/TrafficSync`

Contents

Abstract **i**

1 Introduction and Problem Motivation **1**
1.1 Background . . . 1
1.2 Limitations of fixed-time and independent adaptive control . . . 1
1.3 Neighbour information and the multi-agent view . . . 1
1.4 TrafficSync . . . 2
1.5 Objectives and scope . . . 2
1.6 Contributions . . . 2

2 Problem Formulation and Research Objectives **3**
2.1 Problem statement . . . 3
2.2 Road network and communication network . . . 3
2.3 Agents, observations and actions . . . 3
2.4 Demand, congestion and communication constraints . . . 4
2.5 Performance objectives . . . 4
2.6 Research questions and hypotheses . . . 4

3 Theoretical Foundations and Multi-Agent Systems Concepts **6**
3.1 Graph theory . . . 6
3.2 Distributed consensus . . . 6
3.3 Formation control and green-wave coordination . . . 7
3.4 Resilient consensus . . . 7
3.5 Distributed optimisation . . . 8
3.6 Reinforcement learning and multi-agent reinforcement learning . . . 9
3.7 Distributed model predictive control . . . 9
3.8 Formal methods . . . 9
3.9 Epidemic models and spectral analysis . . . 10

4 Overall System Architecture and Design **11**
4.1 Layers and data flow . . . 11
4.2 The simulation loop . . . 12
4.3 Repository structure . . . 12

5 Mathematical Traffic Network and Simulator **13**

5.1 Road network construction . . . 13
5.2 Vehicle-level point-queue model . . . 13
5.3 Demand . . . 14
5.4 Signal state machine . . . 14
5.5 Validation (E0) . . . 14

6 Information Acquisition and Communication Model 15
6.1 Local detectors . . . 15
6.2 Probe vehicles . . . 15
6.3 Messages . . . 15
6.4 Communication bus . . . 15
6.5 Why imperfections matter . . . 16

7 Traffic Signal Control Algorithms 17
7.1 C0 – Fixed-time Webster control . . . 17
7.2 C1 – Independent adaptive control . . . 17
7.3 C2a – CoMP: neighbour-informed max-pressure . . . 18
7.4 C2b – CoCycle: consensus cycle and formation offsets . . . 18
7.5 CoMP+Cycle . . . 19
7.6 Extensions . . . 19

8 Learning and Formal Verification Extensions 21
8.1 Multi-agent reinforcement learning . . . 21
8.2 Signal Temporal Logic monitoring and constraints . . . 21
8.3 Epistemic safety . . . 22

9 Experimental Methodology and Evaluation Framework 23
9.1 Configurations and protocol . . . 23
9.2 Fair comparison: common random numbers . . . 24
9.3 Parameter tuning . . . 24
9.4 Statistical methods . . . 24
9.5 Metrics . . . 24

10 Results, Visualisation and Detailed Analysis 25
10.1 E0 – Simulator validation . . . 25
10.2 E1 – Baseline sanity . . . 25
10.3 E2 – Main controller comparison (RQ1, H1) . . . 26
10.4 Ablations – where does CoCycle's gain come from? . . . 27
10.5 E3 – Network size (RQ2, H2) . . . 28
10.6 E4 – Demand sweep (RQ2, H2) . . . 29
10.7 E5 – Information quality (RQ4) . . . 29
10.8 E6 – Communication topology (RQ3, H3) . . . 30
10.9 E7 – Communication failures (RQ4, H4) . . . 31
10.10 E8 – Consensus validation . . . 31

10.11E9 – Resilient consensus (RQ4, H5) . . . 32
10.12E10 – Distributed optimisation . . . 32
10.13E11 – Congestion as an epidemic (RQ5, H6) . . . 33
10.14E12 – Multi-agent reinforcement learning . . . 34
10.15E13 – DMPC, STL constraints and epistemic guards . . . 34
10.16E14 – Incident and tidal scenarios . . . 35
10.17Time–space diagram and demonstration . . . 35
10.18SUMO validation and real-city scenarios . . . 36

11 Discussion and Critical Interpretation 38
11.1 When does coordination actually help? . . . 38
11.2 Why CoCycle wins in the queue model . . . 38
11.3 Why CoMP behaves like independent control . . . 38
11.4 Why noisy probe information degrades adaptive control . . . 39
11.5 Why connectivity does not improve performance . . . 39
11.6 Why more complex controllers can perform worse . . . 39
11.7 Overload and spillback . . . 39
11.8 Theory and practice . . . 39
11.9 Hypotheses . . . 40

12 Limitations and Future Work 41
12.1 Limitations . . . 41
12.2 Future work . . . 41

13 Conclusion 43

A Team, Reproducibility and Parameters 44
A.1 Individual contributions . . . 44
A.2 Reproducibility . . . 45
A.3 Software . . . 45
A.4 Default parameters . . . 45
A.5 Consensus on the 3-node path . . . 46
A.6 Demonstration still . . . 46

List of Figures

4.1 TrafficSync architecture and data flow. Highlighted: the simulator and the agents, which only interact through observations, phase requests and the message bus. . 11

10.1 E1 on grid2x2: mean delay per controller and demand level (10 seeds, 95% CI). Actuated and local max-pressure coincide. 26
10.2 E2 (F01): mean delay by scenario and controller with local detectors (left) and 20% probes (right); 30 seeds, 95% CI. 27
10.3 Ablations: mean delay of all CoMP and CoCycle variants (10 seeds, 95% CI). . . 28
10.4 Left: E3 (F03), paired delay change vs actuated by network size. Right: E4 (F02), delay vs entry rate with 95% CI bands. 29
10.5 E5: delay vs probe penetration (log scale); dashed lines are the delays with local detectors. 30
10.6 E6 (F04): CoCycle delay vs $\lambda_2(L)$ (left) and mean degree (right) of 44 communication graphs; dashed: actuated control. 30
10.7 E7 (F05): delay vs message drop probability (left) and latency (right); dashed: actuated control. 31
10.8 E8 (F06): consensus error vs iteration on random graphs; faster decay for larger λ_2. 31
10.9 E9 (F08): with one faulty agent, plain averaging is captured by the attacker; W-MSR stays in the benign interval. 32
10.10 E10: convergence to $z^\star$ (left, F07) and iterations vs λ_2 (right). 33
10.11 E11: estimated $R = \beta\lambda_{\max}(A)/\gamma$ per controller and scenario (left, F11; dashed: threshold 1) and centrality vs attributed spreading (right). 33
10.12 E12: validation delay during training (mean over training seeds). 34
10.13 E13: spillback events per hour by controller and message loss. 35
10.14 E14 (F09): network mean queue around the incident (shaded: incident window). 35
10.15 F10: time–space diagram of the middle eastbound arterial (grid3x3, `D_med`, 1200–1500 s). 36
10.16 Left: the same vehicles and controllers in the queue simulator and in SUMO. Right: delay per controller in the six real cities. 37
10.17 Browser replay of Bologna Pasubio: all five controllers side by side on the same vehicles (`results/SUMO/web/`). 37

A.1 Laplacian consensus on a 3-node path from $x(0) = [2, 5, 8]$. 46
A.2 Side-by-side animation (fixed, actuated, CoCycle) on grid3x3, `D_med`, $t = 1500$ s: links coloured by occupancy, signal heads and messages (`results/demo/`). 46

List of Tables

2.1 Mapping of research questions and hypotheses to experiments and metrics. . . . 5

5.1 Demand scenarios (rate per entry link). 14
5.2 E0: simulator validation. 14

7.1 Status of the controllers. 20

9.1 Experiments (seeds are evaluation seeds; tuning used seeds 100–102). 23

10.1 E2: mean delay [s] over 30 seeds (95% CI half-widths ≤ 1.6 s at low/med/tidal, ≤ 5.1 s at high/ramp). Bold: best per column. 26
10.2 Ablations: delay change vs actuated. 28
10.3 E4: mean delay [s] vs uniform entry rate (10 seeds) and CoCycle change vs actuated. 29
10.4 E5: delay [s] at `D_med` (10 seeds). 29
10.5 E12: delay on held-out seeds 0–9 (mean of 3 trained policies) and change vs the best handcrafted controller. 34
10.6 E13: mean delay [s] / spillback events per hour (5 seeds). Overload = 0.25 veh/s per entry. 34
10.7 Real cities in SUMO: mean delay [s], 3 seeds; best agent in bold. 36

11.1 Verdicts on the hypotheses. 40

A.1 Main simulation and controller parameters. 45

List of Algorithms

1 One time step of an episode (`runner.run_episode`) 12
2 Common switching rule `select_phase` (every $\Delta_d = 5$ s) 17
3 CoCycle agent i at each decision 19

Chapter 1

Introduction and Problem Motivation

1.1 Background

Congestion in cities is concentrated at signalised intersections. Every second of green given to one approach is a second of red for the others, and because links between junctions are short, the consequences of a decision at one junction travel quickly through the network: platoons released downstream arrive at the next junction a travel time later, and a queue that grows beyond the storage of a link backs up into the junction upstream (*spillback*) and blocks movements that are otherwise green. Signal control is therefore a network problem, even though signals are installed, operated and sensed one junction at a time.

1.2 Limitations of fixed-time and independent adaptive control

The classical engineering answer is a *fixed-time* plan: a cycle length and green splits computed offline from design flows, typically with Webster's formula [1], and optionally offsets between neighbouring junctions to create green waves. Fixed-time plans are robust and predictable, but they ignore the stochastic variation of arrivals and become inefficient whenever demand differs from the design flows (peaks, incidents, tidal flows).

Independent adaptive control (vehicle-actuated control, local max-pressure [2]) reacts to the queues measured at the junction. It removes much of the waste of fixed-time control, but each junction optimises against its own, local view. It cannot anticipate a platoon that has just been released upstream, it may release vehicles into a link that is already full downstream, and adjacent junctions may settle into uncoordinated cycles that stop the same vehicles repeatedly.

1.3 Neighbour information and the multi-agent view

Much of the information a junction is missing is available one hop away: the upstream neighbour knows what it has just released, the downstream neighbour knows how full its approach is. Modern infrastructure (connected signal controllers, probe vehicles reporting positions and speeds) makes it plausible to share this information. This leads naturally to a *multi-agent* formulation: every intersection is an autonomous agent with local sensing, local actuation and the ability to exchange messages with its neighbours over a communication graph. Coordination is then not imposed by a central optimiser but has to emerge from distributed algorithms — consensus on shared quantities, formation control for offsets, distributed optimisation, predictive coordination and learning — that are the core of a Multi-Agent Systems course.

1.4 TrafficSync

TrafficSync is a simulation and experimentation platform that implements this view and uses it to answer, with controlled experiments, *when* sharing traffic information between intersections actually helps. Each intersection is an agent that (i) observes its own approaches through local detectors or noisy probe-vehicle data, (ii) receives messages from its neighbours with delay and possible loss, (iii) updates internal state such as consensus estimates, and (iv) chooses its signal phase subject to safety constraints. The platform includes a deterministic traffic simulator, the information and communication models, a family of controllers ranging from fixed-time to learned policies, formal monitors, and an experiment engine with paired statistics.

1.5 Objectives and scope

The objectives were to:

1. build a reproducible network traffic simulator in which every controller sees identical vehicles;
2. model realistic information (sparse, noisy, delayed probe data) and imperfect communication (latency, loss, outages, faulty agents);
3. implement independent and coordinated multi-agent controllers grounded in the course theory (graph Laplacians, consensus, formation control, resilient consensus, distributed optimisation, DMPC, STL, epistemic logic, epidemic models, MARL);
4. evaluate them with a pre-planned experimental design (E0–E14 and ablations) that answers the research questions of Chapter 2; and
5. check the main conclusions in a microscopic simulator (SUMO [3]) and on real-city scenarios.

The scope is two-phase signals on corridor and grid networks (plus the phase plans of the real-city scenarios in SUMO); pedestrian phases, emergency-vehicle priority and MAPPO were planned as optional and were not implemented (Chapter 12).

1.6 Contributions

- A deterministic, vehicle-level traffic simulator with spillback, validated against saturation flow, conservation and Webster's delay formula (Chapter 5).
- Two coordinated controllers, CoMP and CoCycle, in which coordination is implemented with message passing, dynamic average consensus and formation-control offsets, together with resilient (W-MSR), optimisation-based (ADMM, gradient tracking), predictive (DMPC with STL constraints), knowledge-based (epistemic guard) and learned (IQL/VDN/QMIX) variants (Chapters 7 and 8).
- A complete experimental study with paired statistics over 30 seeds for the main comparison and 10 for the sweeps and ablations (Chapter 10).
- A negative but important result: in the queue model the best coordinated controller wins for reasons that ablations show to be *local*, and information quality matters more than information sharing (Chapter 11).
- A SUMO co-simulation that runs the unchanged controllers on the same vehicles, and an agent layer for real city maps (Cologne, Ingolstadt, Bologna) with interactive browser replays of all controllers side by side.

Chapter 2

Problem Formulation and Research Objectives

2.1 Problem statement

Given a signalised road network with stochastic demand, design local decision rules for the signal at each intersection such that, using only local observations and messages from neighbours delivered over an imperfect channel, network-level delay, queues and spillback are reduced relative to fixed-time and independently adaptive control — and determine the conditions under which this is (or is not) possible.

2.2 Road network and communication network

The road network is a directed graph $\mathcal{G}_R = (\mathcal{V}, \mathcal{L})$ whose $N = |\mathcal{V}|$ nodes are signalised intersections and whose links $l \in \mathcal{L}$ are directed road segments. Each link has length ℓ_l, number of lanes n_l, free-flow speed v_f, free-flow travel time $\tau_l = \lceil \ell_l / v_f \rceil$, storage capacity $C_l = \lfloor n_l \ell_l / s_{\text{veh}} \rfloor$ with jam spacing $s_{\text{veh}} = 7.5$ m, and saturation flow $s_l = 0.5\, n_l$ veh/s. Entry links start at the boundary and exit links end there. A *movement* (l, m) is a permitted turn from incoming link l to outgoing link m, with turning ratio r_{lm}.

Agents communicate over an undirected graph $\mathcal{G}_C = (\mathcal{V}, \mathcal{E}_C)$, by default the undirected version of the road adjacency, with adjacency matrix A, degree matrix D and Laplacian $L = D - A$. The communication graph is a design variable: experiments replace it by rings, complete graphs, k-hop graphs and Erdős–Rényi graphs, or augment it with extra links.

2.3 Agents, observations and actions

Agent i controls intersection i with a set of phases $\mathcal{P}_i$; each phase p is a set of non-conflicting movements. At decision epochs (every $\Delta_d = 5$ s) the agent observes

$$o_i(t) = \big(\{\hat{q}_l(t), \hat{n}_l(t), \hat{\lambda}_l(t)\}_{l \in \mathcal{L}_i^{\text{in}}},\ \phi_i(t),\ \sigma_i(t),\ t_i^{\text{in}}(t)\big),$$

the estimated queue $\hat{q}_l$, occupancy $\hat{n}_l$ and arrival rate $\hat{\lambda}_l$ of its own incoming links, its current phase ϕ_i, signal state $\sigma_i \in \{\textsc{green}, \textsc{yellow}, \textsc{allred}\}$ and time in state. It also holds the messages $\mathcal{M}_i(t)$ received so far. Its action is a requested phase $a_i(t) \in \mathcal{P}_i$; a signal state machine converts requests into legal signal states (minimum and maximum green, yellow, all-red). Agents never read the simulator's true state.

2.4 Demand, congestion and communication constraints

Vehicles enter at entry links following Poisson processes whose rates define seven demand scenarios (Table 5.1), choose turns at each junction with probabilities 0.70/0.15/0.15 (through/left/right), and leave at exit links. Congestion appears as long queues, as spillback when n_m reaches C_m, and in the extreme as gridlock. Messages are sent every T_c seconds, delivered after a latency of d steps, dropped independently with probability p_{drop}, suppressed during outages, and only allowed along edges of $\mathcal{G}_C$.

2.5 Performance objectives

The primary objective is the mean delay per vehicle (travel time minus free-flow travel time, Section 9.5); secondary objectives are queue length, throughput, stops, spillback, fairness (maximum red time, Jain index), communication overhead, computation time and STL robustness.

2.6 Research questions and hypotheses

RQ1 Does coordination between neighbouring signals reduce delay, queues and travel time compared with fixed-time and independently adaptive control under identical demand?

RQ2 How do network size, demand level and demand asymmetry affect the benefit of coordination?

RQ3 How does the communication structure (topology, $\lambda_2(L)$, drops, latency) affect performance?

RQ4 How well does coordination degrade under imperfect information: sparse, noisy or delayed probe data, dropped messages, faulty or malicious reports?

RQ5 Can congestion spillback be understood through a spectral, epidemic-style threshold on the road graph?

H1 Coordinated control < independent adaptive < fixed-time in mean delay at medium and high demand.

H2 The advantage of coordination grows with demand and with demand asymmetry, and is small at very low demand.

H3 Performance of consensus-based coordination improves with $\lambda_2(L)$ of the communication graph, with diminishing returns.

H4 Performance degrades gracefully with message loss; at 100% loss coordinated control falls back to (approximately) independent control.

H5 With $F = 1$ faulty reporters, plain averaging is biased, whereas W-MSR on a sufficiently robust graph keeps performance close to the fault-free case.

H6 Agents with larger dominant-eigenvector centrality contribute more to spillback spreading, so prioritising them reduces network-wide gridlock risk.

Table 2.1: Mapping of research questions and hypotheses to experiments and metrics.

RQ	Hyp.	Experiments	Main metrics
RQ1	H1	E1 baseline sanity, E2 main comparison (30 seeds, detectors and probes), ablations, E14, SUMO validation	delay, queue, throughput
RQ2	H2	E3 network size, E4 demand sweep, E2 tidal / ramp scenarios	paired delay change vs actuated
RQ3	H3	E6 communication graphs, E8 consensus validation, E10 optimisation rate	delay vs λ_2, iterations to tolerance
RQ4	H4, H5	E5 information quality, E7 communication failures, E9 resilience, E13 epistemic guard under loss	delay, bias, spillback
RQ5	H6	E11 epidemic analysis and prioritisation	$\hat{\beta}, \hat{\gamma}$, R, centrality correlation

Chapter 3

Theoretical Foundations and Multi-Agent Systems Concepts

This chapter introduces the theory used by TrafficSync. Each section states the definitions, explains the intuition, and points to where the concept is implemented and what was measured.

3.1 Graph theory

Definitions. For an undirected graph with N nodes, the adjacency matrix $A \in \{0,1\}^{N\times N}$ has $a_{ij} = 1$ iff i and j are neighbours; $D = \mathrm{diag}(d_1, \ldots, d_N)$ with degrees $d_i = \sum_j a_{ij}$; the graph Laplacian is

$$L = D - A, \qquad (Lx)_i = \sum_j a_{ij}(x_i - x_j). \tag{3.1}$$

Orienting every edge arbitrarily gives the incidence matrix $B \in \{-1, 0, 1\}^{N\times|\mathcal{E}|}$ with $B_{ie} = +1$ if e leaves i and -1 if it enters i; then $L = BB^\top$, so L is symmetric positive semi-definite, $x^\top Lx = \sum_{(i,j)\in\mathcal{E}}(x_i - x_j)^2 \geq 0$ and $L\mathbf{1} = 0$. Its eigenvalues are $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_N = \lambda_{\max}(L)$.

Algebraic connectivity. $\lambda_2(L) > 0$ iff the graph is connected; larger λ_2 means that the graph has no "bottleneck" cut, and (Section 3.2) that disagreement decays faster. $\lambda_{\max}(L) \leq 2d_{\max}$ bounds the admissible step size of consensus. The adjacency spectrum enters elsewhere: $\lambda_{\max}(A)$ is the epidemic threshold quantity (Section 3.9) and its eigenvector defines eigenvector centrality.

In TrafficSync. `network/graph.py` and `network/spectral.py` build A, D, L, B and the spectral summary (also diameter, mean degree and, for Erdős–Rényi graphs, the expected degree $(N-1)p$) for the road and communication graphs. Unit tests check $L\mathbf{1} = 0$, symmetry, $L \succeq 0$, $L = BB^\top$, $\lambda_2 > 0$ iff connected and the closed-form spectra of paths, rings and complete graphs. For the 3×3 grid road graph $\lambda_2(L) = 1.0$ and $\lambda_{\max}(A) = 2.83$.

3.2 Distributed consensus

Average consensus. Agents hold values $x_i(0)$ and must agree on $\bar{x} = \frac{1}{N}\mathbf{1}^\top x(0)$ using only neighbour communication. The discrete-time Laplacian iteration is

$$x(k+1) = (I - \epsilon L)\,x(k) = Wx(k), \qquad x_i(k+1) = x_i(k) + \epsilon\sum_j a_{ij}(x_j(k) - x_i(k)). \tag{3.2}$$

Since $\mathbf{1}^\top W = \mathbf{1}^\top$, the sum (hence the average) is invariant. The eigenvalues of W are $1 - \epsilon\lambda_k$; the disagreement $e(k) = x(k) - \bar{x}\mathbf{1}$ therefore decays as

$$\|e(k)\| \le \rho^k \|e(0)\|, \qquad \rho = \max\left(|1 - \epsilon\lambda_2|,\ |1 - \epsilon\lambda_{\max}|\right), \tag{3.3}$$

which is < 1 iff $0 < \epsilon < 2/\lambda_{\max}(L)$. We use $\epsilon = 1/(d_{\max} + 1)$, which satisfies the bound for every graph, or Metropolis weights $w_{ij} = 1/(1 + \max(d_i, d_j))$ that are doubly stochastic.

Dynamic average consensus. When the local inputs $u_i(k)$ change over time (measured flows), agents track the time-varying average with [4]

$$x(k+1) = Wx(k) + (u(k) - u(k-1)), \qquad x(0) = u(0), \tag{3.4}$$

which keeps $\mathbf{1}^\top x(k) = \mathbf{1}^\top u(k)$ and converges to the moving average when inputs change slowly. "Static" consensus restarts (3.2) from u at each decision.

Why agents agree. CoCycle needs a common cycle length so that offsets are meaningful; CoMP uses a network load estimate $\bar{\rho}$ to weight local pressure. Both are averages of local quantities and are therefore natural consensus problems.

Tested. E8 verifies the path example $[2, 5, 8] \to [5, 5, 5]$, sum conservation to 4.6×10^{-14}, convergence for $\epsilon = 0.99 \cdot 2/\lambda_{\max}$ and divergence for $1.01 \cdot 2/\lambda_{\max}$, a Spearman correlation of -0.99 between iterations to 10^{-6} and λ_2 over 24 random graphs, and a $9\times$ smaller tracking error of dynamic vs restarted consensus (Section 10.10).

3.3 Formation control and green-wave coordination

In formation control, agents keep prescribed relative positions $x_i - x_j = r_i - r_j$ instead of agreeing on one value. Writing $\xi_i = x_i - r_i$ reduces the problem to consensus on ξ; pinning a leader ($b_0 > 0$) fixes the absolute position:

$$\xi_i(k+1) = \xi_i(k) + \epsilon\Big[\sum_j a_{ij}(\xi_j(k) - \xi_i(k)) + b_i(\xi^\star - \xi_i(k))\Big]. \tag{3.5}$$

For signals, the "position" is the cycle offset $\phi_i \in [0, C)$, the time at which the main green starts within the common cycle C. A green wave along a link from i to j requires $\phi_j - \phi_i \equiv \tau_{ij} \pmod C$, the travel time. Because offsets live on a circle, differences are wrapped to $(-C/2, C/2]$:

$$\phi_i \leftarrow \phi_i + \epsilon\Big[\textstyle\sum_j a_{ij}\,\mathrm{wrap}_C\big((\phi_j - \phi_i) - (r_j - r_i)\big) + b_i(\phi_0 - \phi_i)\Big], \tag{3.6}$$

with the reference differences $r_j - r_i$ given by flow-weighted travel times in both directions, agent 0 pinned as leader and the update clipped to 2 s per round. This is the offset layer of CoCycle (Section 7.4); its effect is measured by the "no offsets" ablation and the time–space diagram (F10).

3.4 Resilient consensus

Fault models. A faulty agent may report arbitrary values. Under the F-total model at most F agents in the network are faulty; under the F-local model at most F in any neighbourhood. Plain averaging (3.2) has no defence: a single agent that keeps reporting a constant drags every benign agent to that constant.

W-MSR. In the Weighted Mean-Subsequence-Reduced algorithm [5] each agent sorts the values received from neighbours, removes up to F values strictly larger and F strictly smaller than its own, and applies a convex update with the rest. Benign agents then never leave the convex hull of benign initial values.

Robustness. A set S is r-reachable if some node in S has at least r neighbours outside S. A graph is (r, s)-robust if for every pair of disjoint non-empty sets S_1, S_2 at least one of: all of S_1 is r-reachable, all of S_2 is r-reachable, or at least s nodes of $S_1 \cup S_2$ are r-reachable. W-MSR reaches benign consensus under the F-total model iff the graph is $(F+1, F+1)$-robust, and under the F-local model if it is $(2F+1)$-robust. Road graphs are sparse, so robustness has to be checked and, if necessary, created by adding communication links.

In TrafficSync. The module `control/wmsr.py` implements W-MSR together with a fault injector, and the module `network/robustness.py` computes robustness exactly by enumerating all 3^N assignments of nodes to (S_1, S_2, rest) with bitmasks for $N \leq 12$ (randomised search above), and a greedy augmentation routine adds communication links until a robustness target holds. E9 (Section 10.11) evaluates all of this.

3.5 Distributed optimisation

Cycle-agreement problem. Every agent prefers its own Webster cycle $C_i^\star$ with weight w_i (its critical flow ratio). The network should choose one common cycle:

$$\min_C \sum_{i=1}^{N} f_i(C), \qquad f_i(C) = \tfrac{1}{2} w_i \, (C - C_i^\star)^2, \qquad z^\star = \frac{\sum_i w_i C_i^\star}{\sum_i w_i}. \tag{3.7}$$

The closed form is a weighted average, which links the problem to consensus and gives an exact reference for every solver.

DGD. $x_i(k+1) = \sum_j w_{ij} x_j(k) - \alpha \nabla f_i(x_i(k))$ [6]. With constant α the fixed point satisfies a perturbed optimality condition and is biased; a diminishing step removes the bias slowly.

Gradient tracking. Each agent tracks the average gradient with an auxiliary variable, $x(k+1) = Wx(k) - \alpha y(k)$, $y(k+1) = Wy(k) + \nabla f(x(k+1)) - \nabla f(x(k))$ [7, 8], which converges exactly with a constant step.

ADMM. Consensus ADMM [9] splits the problem into local copies x_i and a global z:

$$x_i^{k+1} = \arg\min_x \; f_i(x) + \tfrac{\rho}{2}(x - z^k + u_i^k)^2 = \frac{w_i C_i^\star + \rho(z^k - u_i^k)}{w_i + \rho}, \tag{3.8}$$

$$z^{k+1} = \tfrac{1}{N} \textstyle\sum_i (x_i^{k+1} + u_i^k), \qquad u_i^{k+1} = u_i^k + x_i^{k+1} - z^{k+1}, \tag{3.9}$$

with primal residual $r^k = \|x^k - z^k \mathbf{1}\|$ and dual residual $s^k = \rho\sqrt{N}|z^k - z^{k-1}|$. The z-update needs a global average; the course "hack" replaces it by K rounds of consensus, and a decentralised edge-based ADMM removes z altogether. All variants are in `control/admm.py` and `control/dgd.py`; E10 compares them (Section 10.12).

3.6 Reinforcement learning and multi-agent reinforcement learning

A Markov decision process $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$ is solved by the action-value function $Q^\star(s,a) = \mathbb{E}[r + \gamma \max_{a'} Q^\star(s',a')]$; deep Q-learning [10] fits Q_θ to the target $y = r + \gamma Q_{\bar\theta}(s', \arg\max_{a'} Q_\theta(s',a'))$ with a target network $\bar\theta$ and, in double Q-learning [11], action selection by the online network. In a multi-agent setting:

- **IQL** [12]: each agent learns $Q_i(o_i, a_i)$ treating others as part of the environment; simple and scalable but non-stationary, because the other agents keep changing.
- **Parameter sharing**: all agents use one network with an agent one-hot in the observation; sample efficiency improves and homogeneous intersections share experience.
- **VDN** [13]: $Q_{tot} = \sum_i Q_i(o_i, a_i)$ trained on the team reward, addressing credit assignment.
- **QMIX** [14]: $Q_{tot} = g_\psi(Q_1, \ldots, Q_N; s)$ with a mixing network monotone in each Q_i, so the decentralised greedy actions remain consistent with the joint greedy action.

VDN and QMIX are *centralised training with decentralised execution* (CTDE): the mixer uses the global state during training only. Details and results are in Sections 8.1 and 10.14.

3.7 Distributed model predictive control

MPC solves, at each step, a finite-horizon optimal control problem from the current state, applies the first control and repeats (receding horizon). In distributed MPC [15], each agent solves a local problem whose disturbances (arrivals) are the *predicted trajectories* (departures) of its neighbours, exchanged by message. For a link l with queue q_l, predicted arrivals $\hat a_l[k]$ and green fraction $g_p[k]$ of the phase serving it,

$$q_l[k+1] = q_l[k] + \hat a_l[k] - d_l[k], \qquad 0 \le d_l[k] \le s_l \,\Delta\, g_{p(l)}[k], \quad d_l[k] \le q_l[k] + \hat a_l[k], \tag{3.10}$$

with $\sum_p g_p[k] \le 1$. This is a quadratic program (QP) solved with CVXPY [16]/OSQP [17]. Its departures d_l become the arrival forecast of the downstream agent. Section 7.6 gives the objective and consistency scheme actually used.

3.8 Formal methods

Signal Temporal Logic. STL [18] formulas over real-valued traces are built from predicates $\mu \equiv x \le c$, Boolean connectives and bounded temporal operators $\Box_{[a,b]}$ (always) and $\Diamond_{[a,b]}$ (eventually). Quantitative robustness [19] assigns a real number whose sign gives satisfaction:

$$\begin{aligned} \rho(x \le c, t) &= c - x(t), \qquad \rho(\neg\varphi) = -\rho(\varphi), \qquad \rho(\varphi \wedge \psi) = \min(\rho_\varphi, \rho_\psi), \\ \rho(\Box_{[a,b]}\varphi, t) &= \min_{t' \in [t+a, t+b]} \rho(\varphi, t'), \qquad \rho(\Diamond_{[a,b]}\varphi, t) = \max_{t' \in [t+a, t+b]} \rho(\varphi, t'). \end{aligned} \tag{3.11}$$

For example $x = 6$ gives $\rho(x \le 10) = 4$ (a unit test). TrafficSync monitors spillback safety $\Box(n_l \le C_l - \delta)$, a response property (a queue above $0.6C_l$ falls below $0.2C_l$ within 180 s) and starvation freedom (red $\le R_{\max} = 120$ s).

Epistemic logic. A Kripke structure $(W, \{\sim_i\}, V)$ has worlds W, an indistinguishability relation $\sim_i$ per agent and a valuation V [20]. Agent i *knows* φ at w, written $K_i\varphi$, iff φ holds in every world $w' \sim_i w$. Equivalently, the information set $I_i(w) = \{w' : w' \sim_i w\}$ must

be contained in the truth set of φ. In TrafficSync the worlds are possible occupancies of a downstream link: a message reporting $\hat{n}$ that is α seconds old is consistent with every n in $I_i = \{n : |n - \hat{n}| \le \delta_0 + \lambda^{\max}\alpha\}$, so knowledge *shrinks with message age*. The epistemic guard allows a movement only if $K_i\,\mathrm{Room}_m$ holds (Section 7.6).

3.9 Epidemic models and spectral analysis

In an SIS model on a graph, an infected (congested) node infects each neighbour at rate β and recovers at rate γ. The mean-field (NIMFA) dynamics are

$$\dot{p}_i = (1 - p_i)\Big(\beta_0 + \beta \sum_j a_{ij} p_j\Big) - \gamma p_i, \tag{3.12}$$

with an added background rate β_0 for congestion that starts locally. Linearising around $p = 0$ (with $\beta_0 = 0$) shows that the congestion-free state is stable iff

$$R \;=\; \frac{\beta\,\lambda_{\max}(A)}{\gamma} \;<\; 1, \tag{3.13}$$

the spectral epidemic threshold [21, 22]. Nodes with large entries of the dominant eigenvector of A (eigenvector centrality) are expected to contribute most to spreading. In TrafficSync an intersection is "infected" while its most loaded approach exceeds half its capacity for more than 60 s; β_0, β, γ are estimated by maximum likelihood from simulated event sequences (`analysis/epidemic.py`); E11 tests the threshold and the centrality hypothesis (Section 10.13).

Chapter 4

Overall System Architecture and Design

4.1 Layers and data flow

TrafficSync is a Python package (`src/trafficsync`) organised in layers with one-directional dependencies (Figure 4.1). Configuration is declarative (YAML); the simulator never imports controllers; controllers see only observations and messages; experiments drive episodes and write results.

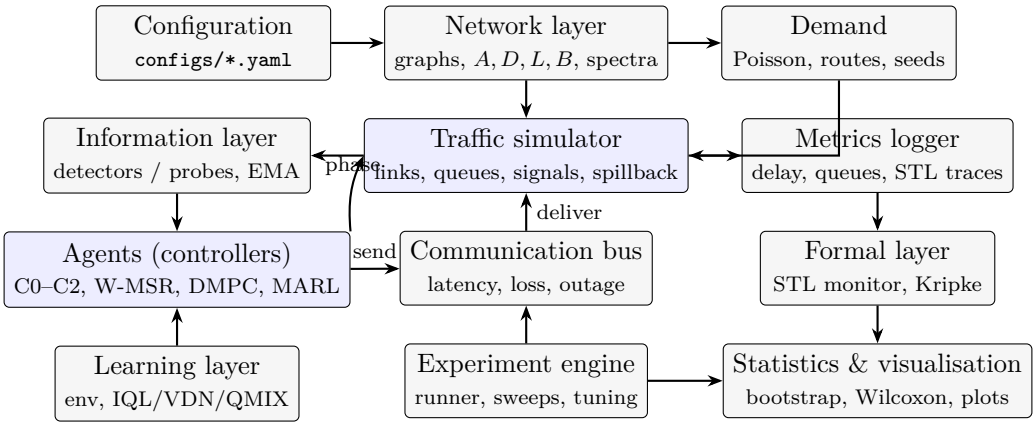


Figure 4.1: TrafficSync architecture and data flow. Highlighted: the simulator and the agents, which only interact through observations, phase requests and the message bus.

Configuration YAML files for the simulator, probe model, communication, demand scenarios, default and tuned controller parameters, and one experiment file per experiment. Controller variants for ablations are declared with a `base` key and overrides.

Network layer (`network/`) builds corridor and grid road graphs, communication graphs, matrices, spectra and robustness checks.

Simulator (`sim/`) moves vehicles, maintains queues, enforces signal timing and spillback, generates demand and records metrics.

Information layer (`sim/probe.py`, `sim/observation.py`) turns true link states into each agent's local estimates.

Communication (`comm/`) defines the message format and the bus.

Agents (`control/`) implement all controllers behind one interface.

Learning (`marl/`) wraps the simulator as a multi-agent environment and trains IQL, VDN and QMIX.

Formal layer (`logic/`) computes STL robustness and knowledge in Kripke models.

Experiment engine (`runner.py`, `experiments/`) runs episodes in parallel, sweeps parameters, tunes on separate seeds and writes `runs.csv`, `summary.md` and figures.

Statistics and visualisation (`analysis/`, `viz/`) compute confidence intervals, paired tests and plots, animations and time–space diagrams.

SUMO co-simulation (`sumo/`) runs the same controllers in SUMO and on real city maps.

4.2 The simulation loop

Algorithm 1 shows one simulated second, executed 3600 times per episode (metrics from $t \geq 600$ s).

Algorithm 1 One time step of an episode (`runner.run_episode`)

$\mathcal{M}_i \leftarrow$ BUS.DELIVER(t) for every agent ▷ messages whose latency has elapsed
for each agent i **do**
  $a_i \leftarrow$ CONTROLLER$_i$.ACT$(o_i(t), \mathcal{M}_i, t)$ ▷ update consensus/state; decide every 5 s
end for
for each agent i **do**
  send the messages of CONTROLLER$_i$.outbox$(o_i(t), t)$ through the bus ▷ drops, topology
end for
SIGNALSTATEMACHINE.UPDATE(a) ▷ min/max green, yellow, all-red
insert vehicles generated at t (Poisson, pre-sampled routes); boundary buffers if entry links are full
move travelling vehicles that reached their stop line into the FIFO queue
discharge queues on effective green with saturation credit, if the next link has room (else spillback)
update probe reports and local estimates $\hat{q}, \hat{n}, \hat{\lambda}$; build $o_i(t+1)$
record metrics (queues, delays, stops, spillback, messages, STL traces); $t \leftarrow t+1$

4.3 Repository structure

```
trafficsync/
  configs/ networks, simulator, probe, communication, demand, controllers, experiments
  src/trafficsync/
    network/ sim/ comm/ control/ marl/ logic/ analysis/ viz/ sumo/ (package)
    config.py runner.py
  experiments/ run_experiment, sweep, tune, core_experiments, e9..e14, make_figures,
               make_demo, sumo_experiments, sumo_tune, make_web_viewer, import_osm_city
  tests/ 166 pytest tests and the regression reference
  results/ E0..E14, ABL, TUNING, SUMO, demo, figures (CSV, summaries, plots)
  sumo_scenarios/ real-city SUMO scenarios with licences
```

Chapter 5

Mathematical Traffic Network and Simulator

5.1 Road network construction

`corridor(n)` builds a chain of n intersections with 400 m links; `grid(r,c)` an $r \times c$ lattice with 300 m links (corridor3, grid2x2, grid3x3, grid4x4, grid5x5). Every intersection has four approaches; boundary approaches get entry and exit links. Links have one lane by default, free-flow speed $v_f = 13.9$ m/s (50 km/h), free-flow time $\tau_l = \lceil \ell_l / v_f \rceil$ (22 s for 300 m), storage $C_l = \lfloor n_l \ell_l / 7.5 \rfloor$ (40 vehicles for 300 m) and saturation flow $s_l = 0.5 n_l$ veh/s (1800 veh/h/lane).

5.2 Vehicle-level point-queue model

The simulator is discrete in time ($\Delta t = 1$ s) and continuous in vehicles. Each link holds two FIFO structures (`sim/link.py`): a *travel* deque of (arrival time, vehicle) pairs for vehicles driving along the link, and a *queue* at the stop line. A vehicle entering link l at time t joins the queue at $t + \tau_l$. The link occupancy is $n_l = |\text{travel}_l| + |\text{queue}_l|$ and the queue length is $q_l = |\text{queue}_l|$.

Discharge with saturation credit. When a movement of link l has effective green (green minus start-up lost time of 2 s), the link accumulates credit $c_l \leftarrow c_l + s_l \kappa_l(t)$, where $\kappa_l(t) \leq 1$ is a capacity factor (reduced during incidents). The head vehicle leaves when $c_l \geq 1$ *and* its next link m has room ($n_m < C_m$, measured once per step before any discharge, so results do not depend on processing order); each departure consumes one unit of credit. If the next link is full the vehicle stays (*spillback*) and blocks the queue behind it (FIFO). While the queue is empty during effective green, up to one vehicle of credit is banked, so a lone vehicle arriving on green passes without waiting.

Why point queues. The model reproduces what matters for signal control — saturation discharge, stop-line queues, storage limits and blocking — at a cost of a few microseconds per vehicle-step, which made 30-seed experiments with thousands of episodes feasible. Car-following, lane changing and turning conflicts are not modelled; the SUMO co-simulation (Section 10.18) checks which conclusions survive them. Compared with the cell transmission model [23] it tracks individual vehicles, so delay, stops and routes are exact per vehicle.

5.3 Demand

Arrivals at each entry e are Poisson with rate $\lambda_e(t)$; each vehicle's route is sampled at creation from turning probabilities 0.70/0.15/0.15. All draws come from random streams seeded by (seed, entry), so **every controller sees identical vehicles, arrival times, routes and probe flags for a given seed**. If an entry link is full, vehicles wait in an unbounded boundary buffer; they are counted in delay, so demand is never silently lost.

Table 5.1: Demand scenarios (rate per entry link).

`D_low`	0.05 veh/s on every entry
`D_med`	0.12 veh/s
`D_high`	0.20 veh/s (near saturation of a one-lane, two-phase junction)
`D_tidal`	east–west 0.22, north–south 0.06 (asymmetric rush hour)
`D_ramp`	0.05 → 0.20 over 0–1200 s, held until 2400 s, back to 0.05 at 3600 s
`D_incident`	0.12; the central eastbound link keeps 30% of its capacity during 1200–2100 s
`D_hotspot`	0.08 background and one entry at 0.35 veh/s
number	uniform rate on every entry (E4 sweep, overload 0.25)

5.4 Signal state machine

Each intersection has two phases (N–S and E–W, each serving through, right and permissive left movements). The state machine (`sim/intersection.py`) runs GREEN→YELLOW (3 s)→ALLRED (1 s)→GREEN, enforces $g_{\min} = 5$ s and $g_{\max} = 40$ s, and never serves conflicting movements together. Controllers only *request* phases; a request is honoured when it is legal. Time since last green (`tsg`) is tracked for the starvation property.

5.5 Validation (E0)

Table 5.2: E0: simulator validation.

Saturation flow	continuous green after a 400 s red: measured 0.500 / 1.000 / 1.500 veh/s for 1 / 2 / 3 lanes (0.0% error)
Conservation	generated = in network + completed, $q_l \geq 0$, $n_l \leq C_l$, checked at 75,600 network-steps (3 networks × 7 scenarios): 0 violations
Webster delay	isolated intersection, 60 s cycle: simulated 12.5 / 14.1 / 16.1 / 22.6 s vs Webster 12.3 / 14.2 / 16.6 / 22.3 s at degree of saturation $x = 0.2/0.4/0.6/0.8$ (-3.2% to $+1.2\%$)
Determinism	same seed twice: identical simulation metrics; fixed, actuated and CoCycle see the same 5,145 vehicles and routes (seed 7)

Webster's uniform-plus-random delay formula $d = \frac{C(1-g/C)^2}{2(1-x\,g/C)} + \frac{x^2}{2\lambda(1-x)} - 0.65\left(\frac{C}{\lambda^2}\right)^{1/3} x^{2+5g/C}$ is itself an approximation, so agreement within a few percent across the whole range of x is strong evidence that queueing and discharge are modelled correctly. An earlier version started the discharge credit at zero on every green second, which made every vehicle wait one second at an empty stop line; this was found during the final verification, fixed (credit banking above), and every experiment was re-run after the fix.

Chapter 6

Information Acquisition and Communication Model

6.1 Local detectors

In the `perfect_sensor` mode an agent reads the true queue q_l and occupancy n_l of its own incoming links, and an arrival rate estimate $\hat{\lambda}_l$ obtained by counting entries into the link and smoothing them with an exponential moving average (EMA). It never sees neighbours' links directly: anything beyond its approaches must arrive by message.

6.2 Probe vehicles

In the `probe` mode only a fraction π of vehicles (the penetration rate, default 0.2) report. Every 5 s each probe reports its speed (Gaussian noise, $\sigma_v = 1$ m/s) and position (noise 5 m); reports arrive with a delay of 2 s. The agent estimates

$$\hat{q}_l = \frac{1}{\pi}\#\{\text{probes on } l \text{ with reported speed} < v_{\text{stop}}\} \cdot \eta, \qquad \hat{n}_l = \frac{1}{\pi}\#\{\text{probes on } l\}, \tag{6.1}$$

with a correction factor η for moving probes that are misread as stopped because of noise, and smooths both with $\hat{x} \leftarrow (1-\alpha)\hat{x} + \alpha x$, $\alpha = 0.3$. Probe flags are sampled per vehicle from the demand stream, so the same vehicles are probes for every controller. The estimates are unbiased in expectation but very noisy at low π: with $\pi = 0.2$ a queue of 5 vehicles contains on average one probe.

6.3 Messages

A `Message` (`comm/messages.py`) carries sender, receiver, send time and any of: current phase and time to the next possible switch, queues, occupancies and arrival rates of the sender's links, predicted departures (platoons), consensus values, cycle offset, DMPC plans and free-form extras. `size_bytes()` estimates the wire size (4 bytes per number) for the overhead metric.

6.4 Communication bus

The bus (`comm/bus.py`) delivers each message after `latency_steps` (default 1 s), drops each message independently with probability p_{drop} (seeded), drops everything inside an outage window, and rejects messages between agents that are not neighbours in $\mathcal{G}_C$ (`enforce_topology`). Agents

send every $T_c = 5$ s. Receivers keep the latest message per sender together with its age, so controllers can extrapolate (CoMP) or quantify uncertainty (epistemic guard). The logger reports messages and bytes per agent per second.

6.5 Why imperfections matter

Real deployments face exactly these effects: radio links lose packets, back-end systems add latency, and probe data cover only connected vehicles. A coordination scheme that only works with perfect, instantaneous information is not useful in practice. The experiments therefore vary penetration, noise and delay (E5), drop rate, latency, period and outages (E7), and faulty reports (E9), and the controllers are designed to fall back to local behaviour when information is missing or stale.

Chapter 7

Traffic Signal Control Algorithms

All controllers implement one interface (`control/base.py`) with four methods: receive the inbox, update internal state before a decision, decide, and produce the outbox. Every adaptive controller ends with the same switching rule (Algorithm 2), so they differ only in the *score* they assign to phases and in what they communicate. Parameters were tuned on separate seeds (100–102) that are never used for evaluation (Section 9.3).

Algorithm 2 Common switching rule `select_phase` (every $\Delta_d = 5$ s)

Input: current phase c, time in green e, phase scores $S(p)$, margin θ, gap-out flag

if $e < g_{\min}$ **then return** c
end if
$b \leftarrow \arg\max_{p \neq c} S(p)$
if $e \geq g_{\max}$ **then return** b
end if
if $S(b) - S(c) > \theta$ **then return** b
end if
if gap-out and the approaches of c are empty and $S(b) > 0$ **then return** b
end if
return c

7.1 C0 – Fixed-time Webster control

Objective. A robust, non-adaptive reference. **Formulation.** Design flows λ_l are propagated from the entry rates through the turning ratios (never measured at run time). With critical flow ratios $y_p = \max_{l \in p} \lambda_l / s_l$, $Y = \sum_p y_p$ and lost time $L_{tot} = 12$ s per cycle,

$$C_0 = \frac{1.5 L_{tot} + 5}{1 - Y} \in [30, 120] \text{ s}, \qquad g_p = \frac{y_p}{Y}(C_0 - L_{tot}), \tag{7.1}$$

offsets 0. **Information.** None at run time. **Advantages.** Predictable, immune to sensor noise. **Limitations.** Cannot react to random fluctuations, peaks or incidents. **Role.** Baseline "city plan"; the reference in E1/E2.

7.2 C1 – Independent adaptive control

Actuated (queue-greedy). The score of a phase is the turning-weighted queue $S(p) = \sum_{(l,m) \in p} r_{lm} \hat{q}_l$; tuned margin $\theta = 16$ vehicles with gap-out. **Local max-pressure.** The classic max-pressure score [2] weights by saturation flow, $S(p) = \sum_{(l,m) \in p} s_l r_{lm} (\hat{q}_l - \hat{q}_m)$; locally the

downstream queue $\hat{q}_m$ is unknown and set to 0, and $\theta = 8$. With one lane per approach every $s_l = 0.5$, so the two scores differ by a constant factor and the tuned margins make the two controllers take *identical* decisions — confirmed in every experiment. **Information.** Own queues only. **Advantages.** Ends empty greens, serves the longer queue; max-pressure is throughput-optimal in theory. **Limitations.** No anticipation of platoons and no knowledge of downstream space. **Role.** The independent-adaptive baseline every coordinated controller must beat. The tuning showed why the margin matters: with an untuned margin $\theta = 2$ the controller switches on every small fluctuation and is 2.0× worse than fixed-time.

7.3 C2a – CoMP: neighbour-informed max-pressure

Objective. Add the information that max-pressure lacks locally: how full the downstream link is and what is about to arrive. **Formulation.**

$$P_i(p) = \gamma_i \sum_{(l,m)\in p} s_l \, r_{lm}(\hat{q}_l + \kappa \, \hat{a}_l - \tilde{q}_m), \qquad \tilde{q}_m = \hat{q}_m^{\text{rep}} + \hat{\lambda}_m \alpha_m, \tag{7.2}$$

where $\hat{q}_m^{\text{rep}}$ is the queue reported by the agent that owns link m, extrapolated over the message age α_m (ignored beyond 30 s); $\hat{a}_l$ are arrivals announced by the upstream agent within a 15 s horizon; a movement is vetoed if the reported occupancy satisfies $\tilde{n}_m \geq C_m - \delta_{room}$ ($\delta_{room} = 2$); and

$$\gamma_i = \text{clip}\big(1 + \beta_c(\bar{\rho}_i - \rho_i),\, 0.5,\, 1.5\big) \tag{7.3}$$

weights the agent's pressure by the difference between the network load $\bar{\rho}_i$, obtained by dynamic average consensus on local loads ρ_i ($K_c = 5$ rounds per decision), and its own load. **Information.** Own queues, neighbours' queues, platoons and load estimate. **Decision.** Algorithm 2 on P_i with $\theta = 8$. **Tuned parameters.** $\kappa = 0$ (anticipation did not help on tuning seeds). **Advantages.** Without messages it reduces exactly to local max-pressure, so failures degrade gracefully. **Limitations.** The downstream term only matters when downstream links are nearly full. **Role.** The coordinated "reactive" controller.

7.4 C2b – CoCycle: consensus cycle and formation offsets

Objective. Classical coordinated control (common cycle, green waves) achieved without a central plan. **Steps** (Algorithm 3):

1. *Local Webster cycle* $C_i^\star$ from measured flows (300 s averages), weight $w_i = Y_i$.
2. *Common cycle* $z = \sum_i w_i C_i^\star / \sum_i w_i$ by dynamic average consensus (3.4) on two channels ($w_i C_i^\star$ and w_i), $\epsilon = 1/(d_{\max} + 1)$, $K_c = 5$ rounds per decision; quantised to 4 s with 3 s hysteresis and bounded to $[40, 100]$ s.
3. *Local splits*: Webster splits of the common cycle from the agent's own flow ratios.
4. *Offsets*: formation-control update (3.6) with travel-time references and agent 0 as pinned leader.
5. *Actuation*: gap-out on empty approaches and extension while a queue remains, at most 40 s of extension per cycle (tuned; 10 s gave a delay ratio of 0.92 to fixed-time, 40 s gave 0.80).

Algorithm 3 CoCycle agent i at each decision

1: update flow estimates; $C_i^\star, w_i \leftarrow \text{Webster}(\hat{\lambda})$; $u_i \leftarrow (w_i C_i^\star, w_i)$
2: **for** $k = 1..K_c$ **do** $x_i \leftarrow x_i + \epsilon \sum_j (x_j - x_i) + (u_i - u_i^{\text{prev}})$ ▷ received x_j
3: **end for**
4: $C \leftarrow \text{quantise}(x_i[0]/x_i[1])$; $g_p \leftarrow$ Webster splits of C
5: $\phi_i \leftarrow \phi_i + \epsilon\big[\sum_j \text{wrap}_C((\phi_j - \phi_i) - (r_j - r_i)) + b_i(\phi_0 - \phi_i)\big]$
6: follow the plan from $(t - \phi_i) \bmod C$; gap-out if the current approaches are empty; extend while a queue remains and extension budget < 40 s
7: send (x_i, ϕ_i) to neighbours

Advantages. Green waves and a common rhythm; robust to message loss because each agent always has a local plan. **Limitations.** A planned cycle makes lone vehicles wait at low demand; the alternative occupancy-based cycle rule maps load to long cycles. **Role.** The coordinated "planned" controller; its gain and the reasons for it are analysed in Section 10.4.

7.5 CoMP+Cycle

The combination follows the CoCycle plan but decides extensions with the CoMP pressure. It was intended to combine anticipation with green waves. In every experiment it was worse than both parents (+17 to +32% vs actuated in E2), because the pressure term keeps extending greens and the cycle plan has to recover the lost time later.

7.6 Extensions

W-MSR integration. Every consensus channel of CoMP and CoCycle can use W-MSR (`aggregation: wmsr`, $F = 1$) instead of averaging. Because W-MSR discards values, it breaks the sum conservation of dynamic consensus; it must therefore be combined with *static* (restarted) consensus (Section 10.11).

Distributed optimisation of the cycle. ADMM, DGD and gradient tracking solve (3.7) and are evaluated offline on cycle instances measured in the simulator (E10); they are not used for closed-loop control.

Epistemic guard. A wrapper for CoMP or DMPC: a movement into link m is allowed only if $K_i\,\text{Room}_m$ holds, i.e. the information set $I_i = \{n : |n - \hat{n}_m| \le \delta_0 + \lambda^{\max}\alpha\}$ ($\delta_0 = 2$, $\lambda^{\max} = s_m$, α the message age; no message: $I_i = \{0, \dots, C_m\}$) lies inside $\{n \le C_m - \delta\}$.

DMPC. Each agent solves, every 5 s, a QP over $H = 12$ steps of 5 s: queue dynamics with arrivals from the committed departure plans of upstream neighbours, green fractions $u_{\min} \le g \le 1$ with $\sum_p g_p \le 1$, and objective

$$\min \sum_{h,l} (w_{\text{lin}} q_l[h] + w_q q_l[h]^2) + w_{\text{sw}} \sum_h \|g[h] - g[h-1]\|_1, \qquad w_{\text{lin}} = 1,\ w_q = 0.05,\ w_{\text{sw}} = 30. \tag{7.4}$$

A pure quadratic objective $w_q q^2$ made DMPC switch 2.3× as often as CoMP and 3× worse in delay, because q^2 favours fractional splits that ignore the lost time of switching; the delay-like linear term with an L1 switching penalty fixed this. Consistency between neighbours' plans is obtained by *sequential* solving in breadth-first order from agent 0, one stage per second, each agent using the plans already committed by agents earlier in the order (a two-sweep negotiation

mode is optional). The phase is chosen from the planned greens of the next 3 steps with a 1.5-step margin. Options: STL constraints (Section 8.2) and the epistemic guard. Solver: OSQP with a CLARABEL fallback (tolerance 10^{-3}); on failure the local rule is used.

MARL policy. A trained Q-network wrapped as a controller (`control/marl_policy.py`), so learned policies are evaluated with exactly the same simulator, seeds and metrics as the handcrafted ones.

Table 7.1: Status of the controllers.

Implemented and evaluated in closed loop	fixed-time, actuated, local max-pressure, CoMP, CoCycle, CoMP+Cycle, W-MSR variants, DMPC (+STL, +epistemic), CoMP+epistemic, IQL, IQL+comm, VDN, QMIX; SUMO grid and real-city agents
Implemented, evaluated offline	ADMM (central, local-z, decentralised), DGD, gradient tracking (E10)
Implemented, unit-tested only	Kripke model checking (hidden-coin model), STL operators beyond the monitored properties
Not implemented (optional extensions)	MAPPO, emergency-vehicle preemption, 4-phase signals

Chapter 8

Learning and Formal Verification Extensions

8.1 Multi-agent reinforcement learning

Environment. `marl/env.py` wraps the simulator in a PettingZoo-style [24] parallel API. One environment step is one decision interval (5 s). The observation of agent i contains its own normalised queues and occupancies per approach, the current phase (one-hot), a green flag, the normalised time in green and an agent one-hot (for parameter sharing). The variant `iql+comm` adds the downstream occupancies received through the message bus. The action is the requested phase; the signal state machine keeps it legal. The reward is the negative local queue, $r_i = -\sum_{l \in \mathcal{L}_i^{\text{in}}} q_l$; VDN and QMIX train on the team reward $\sum_i r_i$.

Learners. All variants share an MLP with two hidden layers of 64 units, a replay buffer of 10^5 transitions, batch size 64, discount $\gamma = 0.99$, target-network update every 200 gradient steps and double-Q targets. Learning rates are 10^{-3} (IQL) and 5×10^{-4} (VDN, QMIX). VDN sums the agents' chosen Q-values; QMIX mixes them with a hypernetwork conditioned on the global state, constrained to non-negative weights (monotonicity).

Protocol. Training uses random uniform demand $\lambda \sim U[0.05, 0.20]$ veh/s per episode on grid3x3 with ϵ-greedy exploration from 1.0 to 0.05. The greedy policy is validated on seeds 200–201 every few episodes; early stopping is allowed only after ϵ has reached its floor (an earlier version stopped at 40–50 episodes while still exploring). Each variant is trained with 3 training seeds (about 70–90 episodes, 20–26 thousand gradient steps, 2 minutes each), and the best checkpoint is tested on held-out seeds 0–9 in `D_med` and `D_high` (seen demand levels) and `D_tidal`, `D_hotspot` and `D_incident` (unseen patterns). A sensitivity run with learning rate 10^{-4} checks whether VDN/QMIX are limited by optimisation.

8.2 Signal Temporal Logic monitoring and constraints

`logic/stl.py` implements robustness semantics on finite traces (windows are cut at the end of the trace). `TraceMonitor` evaluates on every run:

$$\Phi_{\text{safe}} = \square\,(n_l \le C_l - \delta) \qquad \text{spillback safety on every internal link,} \tag{8.1}$$

$$\Phi_{\text{resp}} = \square(q_l > 0.6C_l \rightarrow \Diamond_{[0,180]}\, q_l < 0.2C_l) \qquad \text{queue clearance,} \tag{8.2}$$

$$\Phi_{\text{starv}} = \square\,(\text{red}_p \le R_{\max}),\ R_{\max} = 120\text{ s} \qquad \text{starvation freedom.} \tag{8.3}$$

The overall robustness $\rho(\Phi)$ is the minimum over properties and links; $\rho < 0$ means a violation and its magnitude says by how much. In DMPC the same properties become linear constraints: $n_m[h] \leq C_m - \delta$ on predicted occupancies (softened with a penalised slack for feasibility), and a phase whose red time would exceed $R_{\max}$ within the horizon must receive at least one full step of green before then.

8.3 Epistemic safety

The epistemic guard (Section 7.6) implements the knowledge condition $K_i\,\mathrm{Room}_m$. It is deliberately conservative: when messages are fresh the information set is narrow and the guard rarely binds; as messages age or are lost the set widens to the whole range $\{0, \ldots, C_m\}$ and the agent no longer *knows* that there is room, so it holds the movement. The guard therefore trades throughput for safety exactly when communication is poor — an effect quantified in E13. `logic/kripke.py` provides general Kripke structures; unit tests check the hidden-coin model (K_a *heads* true for the agent who saw the coin, false for the other) and that knowledge shrinks with message age.

Chapter 9

Experimental Methodology and Evaluation Framework

9.1 Configurations and protocol

Every experiment is defined by a YAML file or a function in `experiments/` and produces `results/<id>/runs.csv` (one row per episode), `summary.md` and figures. Unless stated otherwise: grid3x3, local detectors, 3600 s episodes with metrics from $t = 600$ s, decisions every 5 s, messages every 5 s with 1 s latency and no loss, and tuned controller parameters. Table 9.1 lists the experiments.

Table 9.1: Experiments (seeds are evaluation seeds; tuning used seeds 100–102).

ID	Seeds	Design
E0	–	saturation flow (1/2/3 lanes), conservation in all scenarios, Webster delay, determinism
E1	10	corridor3, grid2x2 × `D_low`/`D_med`/`D_high` × fixed, actuated, local max-pressure
E2	30	grid3x3 × 5 demands × 6 controllers × {detectors, probes $\pi = 0.2$}
E3	10	corridor3, 2×2, 3×3, 4×4, 5×5 × `D_med`/`D_high` × 4 controllers
E4	10	uniform $\lambda \in \{0.03, \ldots, 0.25\}$ (9 levels) × 4 controllers
E5	10	$\pi \in \{0.05, 0.1, 0.2, 0.5, 1\}$, $\pi = 0.2$ without noise / latency / both, detectors
E6	10 / 3	road, ring, complete, 2-hop (10 seeds) and 40 ER graphs $p \in \{0.1, 0.2, 0.4, 0.8\}$ (3 seeds each), CoCycle
E7	10	drop $\{0, \ldots, 1\}$, latency $\{0, \ldots, 10\}$ s, period $\{1, 5, 10, 30\}$ s, 10-min outage; CoMP, CoCycle vs actuated
E8	–	matrix-level consensus validation
ABL	10	16 CoMP / CoCycle variants at `D_med`/`D_high`, reference actuated
E9	10	robustness of graphs, W-MSR vs averaging (all fault placements), closed loop with a faulty centre agent
E10	–	ADMM, DGD, gradient tracking on measured cycle instances; rate vs λ_2
E11	10	epidemic rates per controller and scenario, threshold, centrality, prioritisation
E12	3 train / 10 test	IQL, IQL+comm, VDN, QMIX vs handcrafted controllers
E13	5	DMPC, DMPC+STL, epistemic guards vs CoMP under overload and loss
E14	10 (DMPC 5)	incident recovery and tidal demand
SUMO	5 / 3	grid validation in SUMO (same vehicles); six real-city scenarios

9.2 Fair comparison: common random numbers

Because every random draw that affects traffic comes from streams keyed by (seed, entry), two controllers run with the same seed face *exactly* the same vehicles at the same times on the same routes. Differences between controllers are therefore not confounded by demand noise, and comparisons are *paired*: for each seed s we compute the delay of controller A and of the reference B and analyse the paired differences. This reduces the variance of the comparison by an order of magnitude compared with independent samples (the between-seed variance of delay is much larger than the within-seed difference between controllers).

9.3 Parameter tuning

`experiments/tune.py` evaluates candidate parameters on seeds 100–102, grid2x2 and grid3x3, `D_med`/`D_high`/`D_tidal`, 2400 s episodes, and scores them by the geometric mean of delay relative to fixed-time. Results: actuated $\theta = 16$ with gap-out (score 0.838; $\theta = 2$: 2.02; without gap-out ≥ 1.08); CoMP $\theta = 8$, $\kappa = 0$ (0.836); CoCycle maximum extension 40 s (0.796 vs 0.917 at 10 s); CoMP+Cycle 40 s (1.039). DMPC was tuned on seed 101. Real-city agents were tuned per city on SUMO seed 100 (`experiments/sumo_tune.py`).

9.4 Statistical methods

- **Means and confidence intervals:** mean over seeds with a Student-t 95% interval.
- **Paired percentage change:** $\Delta = 100\,(\bar{x}_A - \bar{x}_B)/\bar{x}_B$ with a 95% *paired bootstrap* interval [25]: seeds are resampled with replacement 5000 times and Δ recomputed. Negative Δ means less delay.
- **Wilcoxon signed-rank test** [26] on paired differences (non-parametric, robust to skewed delays). With n seeds the smallest attainable two-sided p is 2^{1-n}: 1.9×10^{-9} for 30 seeds, 0.002 for 10 and 0.0625 for 5; 5-seed results are therefore judged by their CIs.
- **Holm correction** [27] over the 20 comparisons per information mode in E2.
- **Effect size:** the matched-pairs rank-biserial correlation reported with every Wilcoxon test.

9.5 Metrics

For vehicles v generated after the warm-up, with travel time T_v and free-flow time $T_v^0 = \sum_{l \in \text{route}} \tau_l$:

$$\begin{aligned}
\text{mean delay} &= \tfrac{1}{|V|} \textstyle\sum_v (T_v - T_v^0) \qquad \text{(unfinished vehicles counted with their delay so far)},\\
\text{mean queue} &= \tfrac{1}{T|\mathcal{L}_q|} \textstyle\sum_t \sum_l q_l(t),\\
\text{throughput} &= \#\text{completed trips per hour},\\
\text{stops/veh} &= \text{mean number of times a vehicle joins a stop-line queue or boundary buffer},\\
\text{max red} &= \max_{i,p,t} \text{red}_{i,p}(t),\\
\text{Jain index} &= \big(\textstyle\sum_k x_k\big)^2 / \big(K \sum_k x_k^2\big) \quad \text{over mean waits per (node, phase) [28]},
\end{aligned}$$

plus spillback events (a link reaching capacity), gridlock (in some 300 s window fewer vehicles leave than 5% of those arriving), consensus error $\max_i |x_i - \bar{x}|$ (sampled every 5 s), messages and bytes per agent per second, compute time per decision and STL robustness. In SUMO, delay is SUMO's time loss plus insertion delay of completed trips.

Chapter 10

Results, Visualisation and Detailed Analysis

All numbers below are taken from `results/<id>/summary.md` and the underlying `runs.csv` files. "Δ vs actuated" is the paired percentage change in mean delay with its 95% paired-bootstrap interval; negative is better. Unless stated otherwise: grid3x3 with local detectors.

10.1 E0 – Simulator validation

Question. Is the simulator a trustworthy instrument? **Result.** Table 5.2 (Chapter 5): saturation flow is reproduced exactly for 1–3 lanes, no conservation or capacity violation occurs in 75,600 checked network-steps, simulated delay is within −3.2% to +1.2% of Webster's formula over $x = 0.2$–0.8, and runs are deterministic with identical vehicles across controllers. **Interpretation.** Differences between controllers in the following sections are caused by their decisions, not by numerical artefacts or demand noise.

10.2 E1 – Baseline sanity

Question. Do the independent baselines behave as expected on small networks? **Configuration.** corridor3 and grid2x2, `D_low`/`D_med`/`D_high`, 10 seeds. **Results.** Actuated control beats fixed-time in all six cells, from −2.4% [−3.3, −1.5] (corridor3, `D_low`) to −18.5% [−19.3, −17.8] (grid2x2, `D_med`), all with $p \leq 0.006$ (Figure 10.1). Local max-pressure produces exactly the same delays as actuated control in every cell. **Why.** Actuated control ends empty greens and serves the longer queue; the advantage is smallest at low demand on the corridor, where the fixed plan is rarely wrong, and at high demand, where every approach always has a queue and both strategies are close to saturated cycling. The identity of max-pressure and actuated control follows from equal saturation flows (Chapter 7).

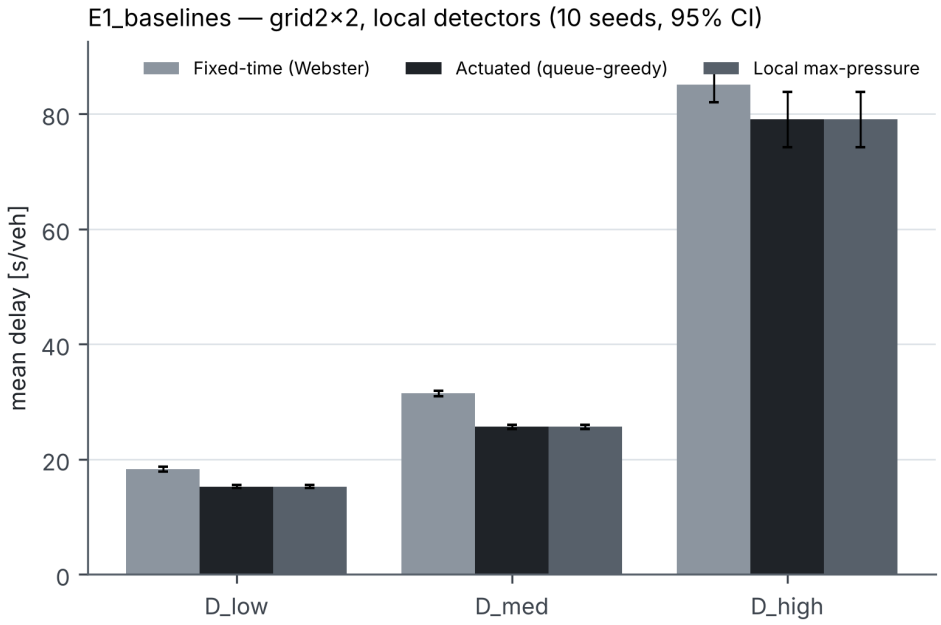


Figure 10.1: E1 on grid2x2: mean delay per controller and demand level (10 seeds, 95% CI). Actuated and local max-pressure coincide.

10.3 E2 – Main controller comparison (RQ1, H1)

Configuration. grid3x3, five demand scenarios, six controllers, 30 seeds, local detectors and 20% probe vehicles; Holm-corrected Wilcoxon tests over 20 comparisons per information mode.

Table 10.1: E2: mean delay [s] over 30 seeds (95% CI half-widths ≤ 1.6 s at low/med/tidal, ≤ 5.1 s at high/ramp). Bold: best per column.

	local detectors					probes $\pi = 0.2$				
controller	low	med	high	tidal	ramp	low	med	high	tidal	ramp
fixed	28.2	47.0	108.8	56.1	151.0	**28.2**	**47.0**	**108.8**	**56.1**	151.0
actuated	**22.4**	37.3	100.1	45.4	**76.5**	42.1	52.1	135.8	64.0	101.3
local max-pressure	**22.4**	37.3	100.1	45.4	**76.5**	42.1	52.1	135.8	64.0	101.3
CoMP	**22.4**	37.3	100.8	45.4	**76.5**	42.1	52.1	131.0	63.9	**99.0**
CoCycle	23.8	**34.8**	**99.4**	**41.8**	76.7	35.3	58.0	132.7	74.3	107.2
CoMP+Cycle	26.9	46.1	117.3	60.0	94.5	35.4	62.3	168.6	89.2	129.6

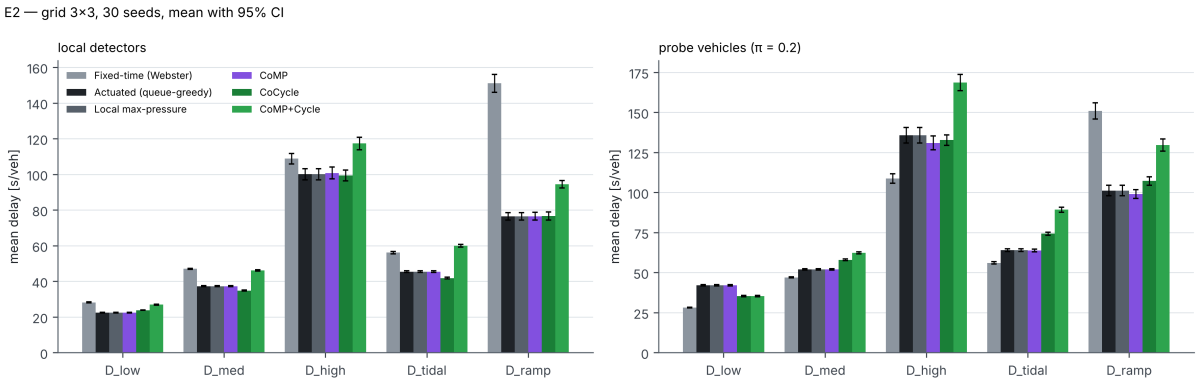


Figure 10.2: E2 (F01): mean delay by scenario and controller with local detectors (left) and 20% probes (right); 30 seeds, 95% CI.

Reading the figure. Each group of bars is one demand scenario; bar height is mean delay and the whiskers the 95% CI over seeds. With detectors the fixed-time bars (grey) are clearly the highest everywhere, and CoCycle is lowest at medium and tidal demand. With probes the ordering is reversed for most scenarios: the grey fixed-time bars are now the lowest.

Observations with local detectors.

- Actuated control beats fixed-time in every scenario, by 8.0% (`D_high`) to 49.3% (`D_ramp`), 20.7% at `D_low` and `D_med`.
- CoCycle beats actuated at `D_med` (−6.7% [−7.1, −6.2]) and `D_tidal` (−8.1% [−8.6, −7.5]), both $p_{\text{Holm}} < 10^{-7}$; it ties at `D_high` (−0.6%, n.s.) and `D_ramp` (+0.2%, n.s.) and loses at `D_low` (+6.6%).
- CoMP equals actuated control (identical at low/med/tidal/ramp, +0.7% n.s. at high).
- CoMP+Cycle is worse than actuated in every scenario (+17 to +32%).

Observations with probes. Every adaptive controller is worse than fixed-time except under `D_ramp` (actuated +10.7% at `D_med`, +24.8% at `D_high` vs fixed). CoCycle beats actuated at `D_low` (−16.1%) but loses at `D_med` (+11.4%) and `D_tidal` (+16.1%); CoMP gains −3.5% [−4.5, −2.5] at `D_high` and −2.2% at `D_ramp`.

Why. (i) Fixed-time control cannot react to random queues, while actuated control ends empty greens. (ii) With sparse, noisy probe estimates adaptive controllers switch on phantom queues and miss real ones; a fixed plan ignores the noise. Under `D_ramp` demand changes so much that a plan designed for the average is badly wrong, and adaptation wins despite noise. (iii) With perfect detectors the CoMP neighbour terms almost never change a decision at the tuned margin; with noisy local estimates the neighbour reports carry real information. (iv) At low demand a planned cycle makes isolated vehicles wait for their phase, which a pure actuated controller avoids. **Hypothesis H1** is therefore supported for CoCycle at medium and tidal demand with detectors — but the ablations below show that this gain is not a coordination effect.

10.4 Ablations – where does CoCycle's gain come from?

Configuration. 16 variants at `D_med`/`D_high`, 10 seeds, reference actuated (Table 10.2, Figure 10.3).

Table 10.2: Ablations: delay change vs actuated.

variant	D_med	D_high
CoCycle (full)	−7.1%	+0.2%
CoCycle, no consensus ($K_c = 0$, own cycle)	−7.4%	−0.2%
CoCycle, no offsets	−7.4%	+0.7%
CoCycle, $K_c = 1$ / $K_c = 10$	−7.4% / −7.1%	+0.1% / +0.2%
CoCycle, occupancy cycle rule	−7.1%	+51.8%
CoMP (full)	0.0%	+1.0%
CoMP, no downstream term	0.0%	−0.3%
CoMP, no γ / $K_c = 0$	0.0%	+1.6%
CoMP, no veto; $\kappa = 0$ (= tuned value)	0.0%	+1.0%
CoMP+Cycle	+23.2%	+18.1%

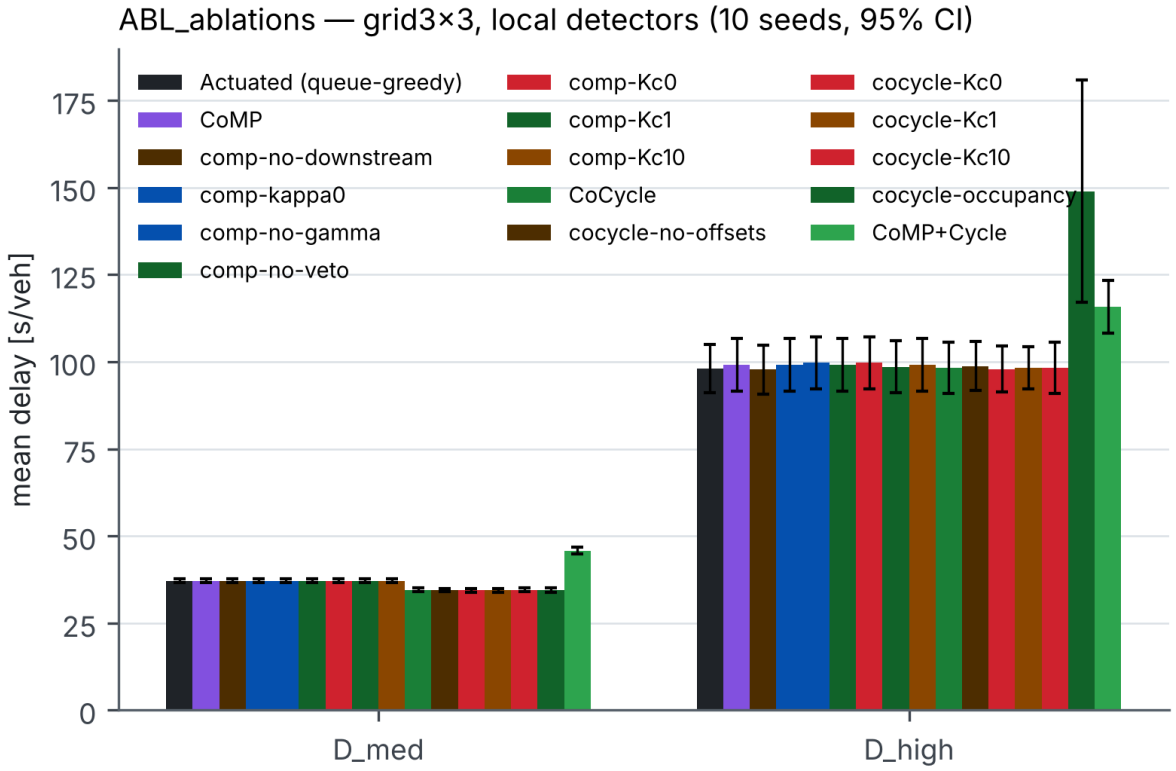


Figure 10.3: Ablations: mean delay of all CoMP and CoCycle variants (10 seeds, 95% CI).

Reading. Removing consensus or offsets leaves CoCycle's gain unchanged. Its gain is therefore produced by the *local* structure — a Webster split plan with a bounded 40 s actuation window — not by agreement or green waves; we did not isolate which part of that local structure matters. The occupancy rule maps high load to long cycles that waste green. For CoMP every component is inert with perfect detectors.

10.5 E3 – Network size (RQ2, H2)

At `D_med` CoCycle reduces delay vs actuated by 8.5% (corridor3), 6.4% (2×2), 7.1% (3×3), 6.9% (4×4) and 6.5% (5×5), with no trend in N (Figure 10.4, left). At `D_high` all sizes are within ±1.3% (n.s.) except corridor3 (−2.8%). CoMP is 0.0% at `D_med` and −0.3 to +2.5% at `D_high`.

Why. A local effect does not depend on how many agents interact. H2 is not supported for network size.

10.6 E4 – Demand sweep (RQ2, H2)

Table 10.3: E4: mean delay [s] vs uniform entry rate (10 seeds) and CoCycle change vs actuated.

λ [veh/s]	0.03	0.05	0.08	0.11	0.14	0.17	0.20	0.23	0.25
fixed	24.5	28.0	32.2	42.2	63.3	73.9	107.7	290.3	433.2
actuated	19.2	22.3	27.9	34.4	43.3	58.8	98.1	269.0	411.5
CoCycle	21.3	23.9	27.5	32.3	40.4	56.3	98.3	252.6	383.3
Δ [%]	+10.7	+7.2	−1.5	−6.0	−6.7	−4.2	+0.2	−6.1	−6.9

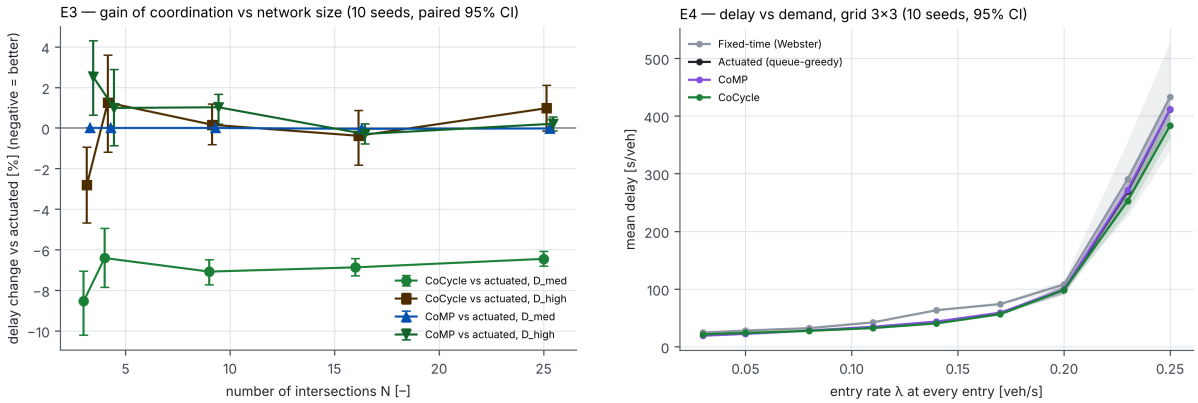


Figure 10.4: Left: E3 (F03), paired delay change vs actuated by network size. Right: E4 (F02), delay vs entry rate with 95% CI bands.

Saturation. Throughput stops increasing between 0.20 and 0.23 veh/s (8521 → 8933 → 8881 veh/h) while delay jumps from about 100 s to 250–430 s: the network saturates. **Pattern.** CoCycle loses at light demand, wins at moderate demand, ties at 0.20 and wins again beyond saturation. The gain is non-monotonic in demand, so H2 is not supported.

10.7 E5 – Information quality (RQ4)

Table 10.4: E5: delay [s] at `D_med` (10 seeds).

information	fixed	actuated	CoMP	CoCycle
probes $\pi = 0.05$	46.9	60.6	60.6	88.2
probes $\pi = 0.1$	46.9	58.3	58.0	71.7
probes $\pi = 0.2$	46.9	52.0	52.0	57.7
probes $\pi = 0.5$	46.9	44.0	44.1	44.2
probes $\pi = 1.0$	46.9	44.3	44.4	41.1
$\pi = 0.2$, no noise	46.9	50.9	50.7	53.0
$\pi = 0.2$, no latency	46.9	55.2	55.0	60.8
local detectors	46.9	37.2	37.2	34.5

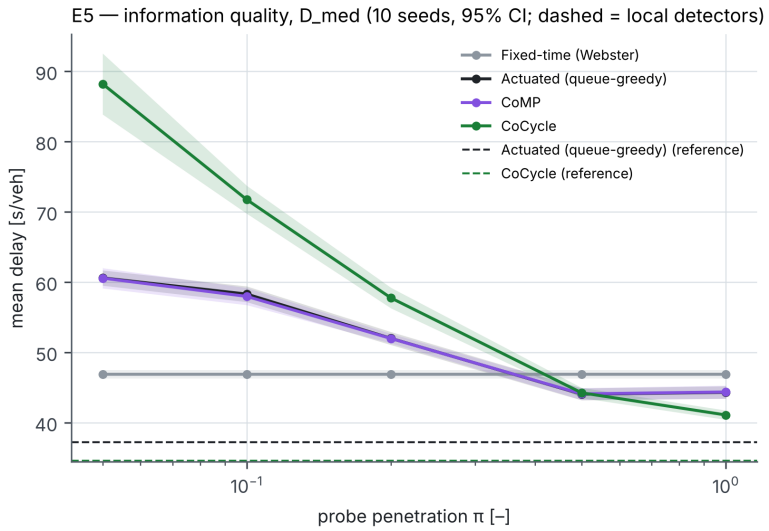


Figure 10.5: E5: delay vs probe penetration (log scale); dashed lines are the delays with local detectors.

Below $\pi = 0.5$ adaptive control is worse than the fixed plan; at $\pi = 1$ CoCycle reaches 41.1 s, still well above the 34.5 s with detectors because probes report only every 5 s, with noise and delay, and the estimator scales and smooths. Removing speed noise helps (CoCycle 57.7 → 53.0 s). Removing the 2 s report delay made results slightly *worse* (actuated 52.0 → 55.2 s); we did not investigate this further. **Conclusion.** The coverage of road-user data matters before any sharing of it.

10.8 E6 – Communication topology (RQ3, H3)

Configuration. CoCycle at `D_med` on 44 communication graphs: road, ring, complete, 2-hop and 40 Erdős–Rényi graphs ($p \in \{0.1, 0.2, 0.4, 0.8\}$, 10 per p; 0/10 connected at $p = 0.1$, 3/10 at $p = 0.2$). **Result.** Delay stays between 34.2 and 34.8 s on every graph, against 37.2 s for actuated control (Figure 10.6); Spearman correlation with λ_2 is +0.19 and with mean degree +0.14. **Why.** λ_2 governs how fast agents agree (E8, E10), but the traffic outcome does not depend on the agreement (ablations). H3 is not supported for traffic performance.

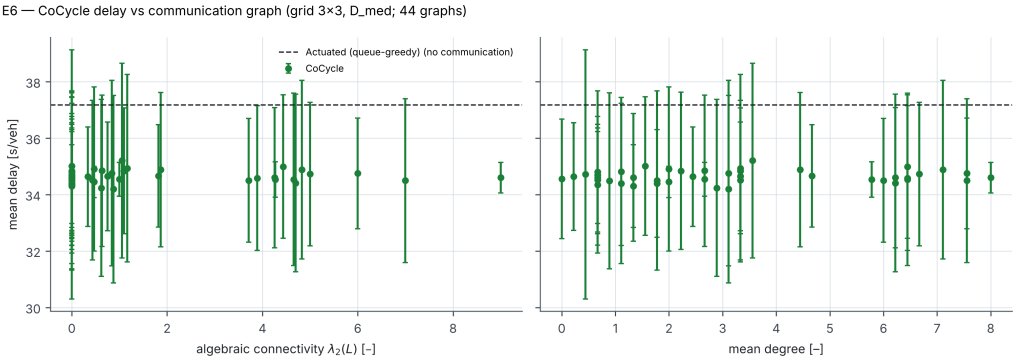


Figure 10.6: E6 (F04): CoCycle delay vs $\lambda_2(L)$ (left) and mean degree (right) of 44 communication graphs; dashed: actuated control.

10.9 E7 – Communication failures (RQ4, H4)

CoMP gives 37.2 s and CoCycle 34.3–34.6 s for every drop probability (0–100%), latency (0–10 s), message period (1–30 s) and during a 10-minute outage (Figure 10.7). H4 is supported, but trivially: nothing degrades because neither controller's performance relies on messages, and with total loss they reduce to their local rules.

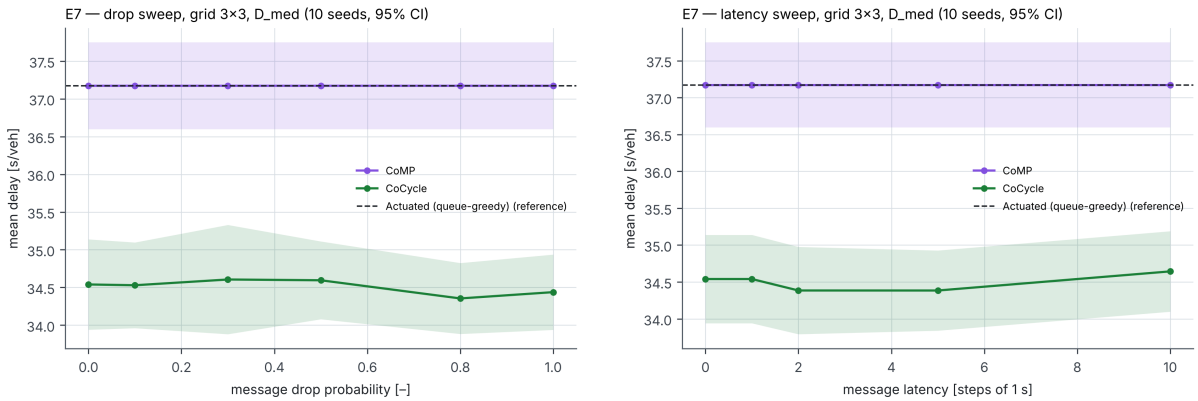


Figure 10.7: E7 (F05): delay vs message drop probability (left) and latency (right); dashed: actuated control.

10.10 E8 – Consensus validation

The 3-node path converges from $[2, 5, 8]$ to $[5, 5, 5]$ with $\mathbf{1}^\top x$ conserved to 4.6×10^{-14}. On a ring of 8, $\epsilon = 0.99 \cdot 2/\lambda_{\max}$ converges (spread 3×10^{-4} after 400 iterations), $1.01 \cdot 2/\lambda_{\max}$ diverges to 2.8×10^{3} and $1.1 \cdot 2/\lambda_{\max}$ to 4.7×10^{31}. Over 24 connected $G(20, p)$ graphs the number of iterations to 10^{-6} correlates with λ_2 with Spearman -0.99 (Figure 10.8). Tracking a drifting average on grid3x3 with $K = 5$, dynamic consensus has error 0.0013 against 0.0120 for restarted static consensus. Every theoretical prediction of Section 3.2 is observed.

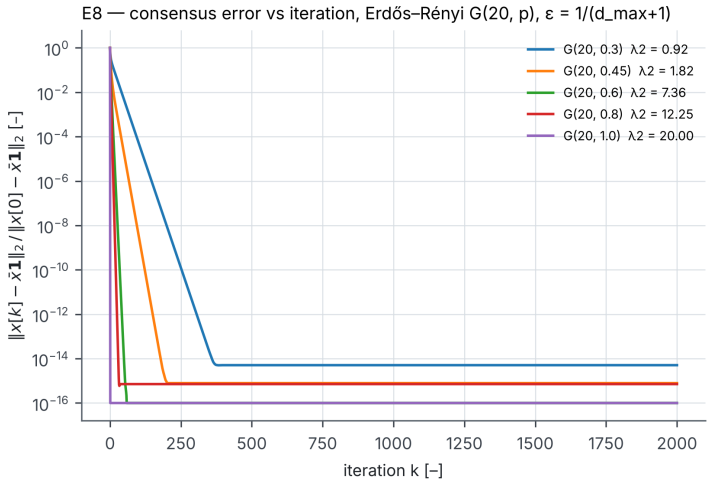


Figure 10.8: E8 (F06): consensus error vs iteration on random graphs; faster decay for larger λ_2.

10.11 E9 – Resilient consensus (RQ4, H5)

Robustness. No road graph is 2-robust. Grid3x3 needs 3 extra links for $(2, 2)$-robustness (exhaustive minimum; greedy augmentation needs 4, a 2-hop graph adds 14) and 8 for 3-robustness; corridor3 and grid2x2 can never be 3-robust, since K_N is only $\lceil N/2 \rceil$-robust. **Matrix level.** Over all placements of one faulty agent, three fault types and 20 initial states, W-MSR keeps benign values inside their initial interval in 100% of runs and plain averaging in 0%; a constant attacker biases plain averaging by 0.62–0.65. On augmented (robust) graphs W-MSR always reaches benign agreement; on the bare road graph only in 15–61% of runs, which illustrates the necessity of the robustness condition (Figure 10.9).

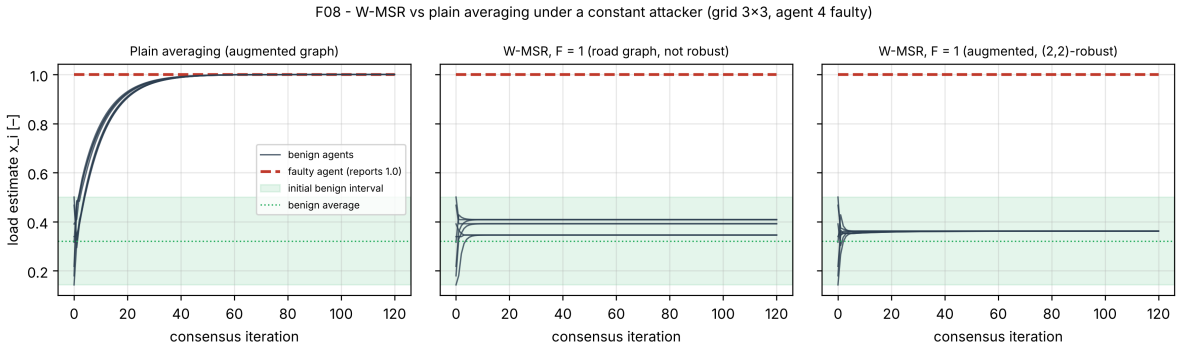


Figure 10.9: E9 (F08): with one faulty agent, plain averaging is captured by the attacker; W-MSR stays in the benign interval.

Closed loop (10 seeds, centre agent faulty). For CoMP at `D_high`, plain averaging increases delay by +18.8% [+17.3, +20.6], W-MSR on static consensus by only +0.6% (n.s.), but W-MSR on *dynamic* consensus by +19.5%: W-MSR discards values and breaks the sum conservation that dynamic tracking relies on, so estimates drift. For CoCycle the attacker pushes benign agents to the maximum cycle (88 s observed) under averaging, while W-MSR keeps it at 47.5 s vs 46.8 s fault-free, yet delay is unchanged (−0.7%, n.s.) — consistent with CoCycle's gain being local. H5 is supported, with two conditions: a robust graph and static consensus.

10.12 E10 – Distributed optimisation

Instances are local Webster cycles from design flows (tidal, hotspot) or from flows measured in one simulated hour; $C_i^\star$ spans up to 36–210 s (with uniform demand all design $C_i^\star$ are equal and the problem is trivial). Central ADMM reaches 10^{-6} in 23–43 iterations, and with the default stopping rule (residuals $< 10^{-3}$) $|z - z^\star| \leq 8.8 \times 10^{-4}$ s. Decentralised ADMM and gradient tracking are exact ($\sim 10^{-13}$) in 98–185 iterations using only neighbour messages. DGD with constant step $\alpha = 0.2$ remains biased by 0.17–20 s. The ADMM "hack" with local z_i has an error of 18 s with one consensus round, 3×10^{-4} with 40 and 8×10^{-9} with 80. Over ER graphs $G(16, p)$ gradient tracking's iterations correlate with λ_2 (Spearman −0.95); decentralised ADMM shows −0.64 with ρ tuned per graph but +0.98 with fixed $\rho = 1$ (Figure 10.10). **Why.** Constant-step DGD converges to a fixed point that is not the optimum; gradient tracking corrects it; ADMM's speed depends on ρ relative to node degrees.

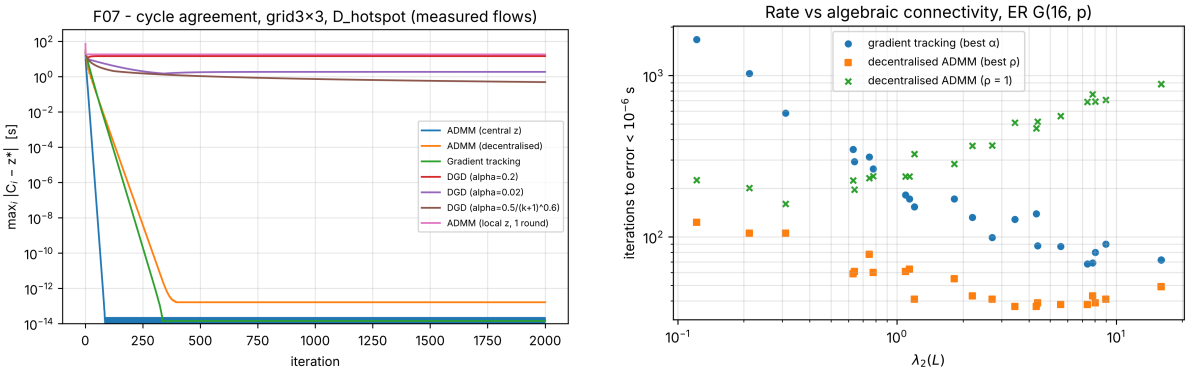


Figure 10.10: E10: convergence to $z^\star$ (left, F07) and iterations vs λ_2 (right).

10.13 E11 – Congestion as an epidemic (RQ5, H6)

With $\lambda_{\max}(A) = 2.83$ for grid3x3, only fixed-time control crosses the threshold: $R = 1.36$ under `D_ramp` and 1.32 under `D_incident` (both $p < 10^{-7}$ for $\beta > 0$), exactly the scenarios with the most spreading (mean congested fraction 0.36 and 0.14). Actuated control keeps $R \leq 0.52$ everywhere, mainly by ending episodes sooner ($1/\gamma$ of 252 vs 774 s and 563 vs 1448 s). At overload (0.25 veh/s) congestion is everywhere but $\hat{\beta} \approx 0$: global saturation, not contagion. Mean-field SIS predicts the observed congested fractions, and eigenvector centrality correlates with attributed spreading (Spearman +0.73). Prioritising the central node through CoMP's weight changed delay by −1.1 to +0.9%, the same as a corner control. The infection definition uses the most loaded approach ($> 0.5C$ for > 60 s): the standard mean-occupancy definition (> 0.7) almost never fired. H6 is partly supported.

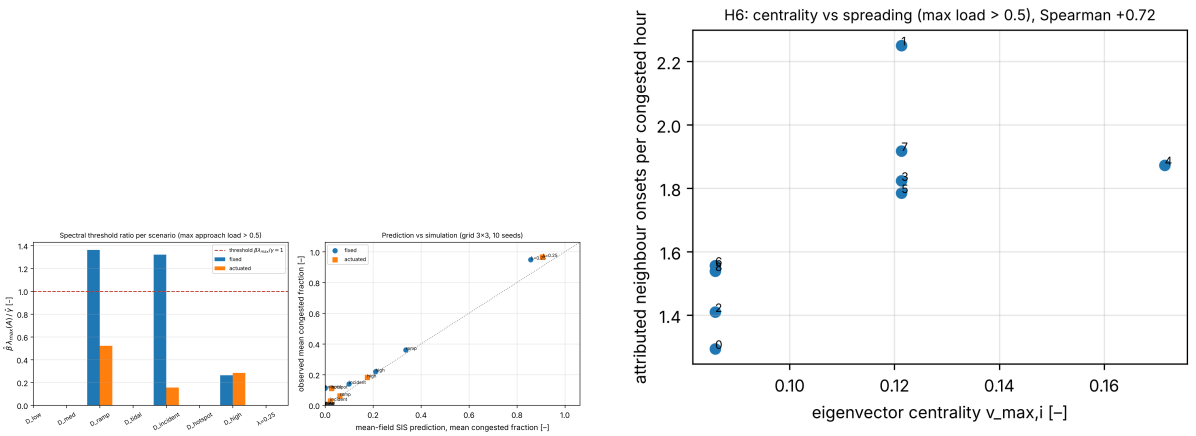


Figure 10.11: E11: estimated $R = \beta\lambda_{\max}(A)/\gamma$ per controller and scenario (left, F11; dashed: threshold 1) and centrality vs attributed spreading (right).

10.14 E12 – Multi-agent reinforcement learning

Table 10.5: E12: delay on held-out seeds 0–9 (mean of 3 trained policies) and change vs the best handcrafted controller.

scenario	best handcrafted	IQL	IQL+comm	VDN	QMIX
`D_med`	34.5 (CoCycle)	40.3 (+17%)	40.5 (+17%)	51.2 (+48%)	88.5 (+156%)
`D_high`	98.1 (actuated)	105.1 (+7%)	110.2 (+12%)	283.4 (+189%)	250.4 (+155%)
`D_tidal` (unseen)	41.7 (CoCycle)	49.4 (+19%)	48.7 (+17%)	57.8 (+39%)	177.4 (+325%)
`D_hotspot` (unseen)	95.9 (CoMP)	105.4 (+10%)	147.3 (+54%)	193.1 (+101%)	303.8 (+217%)
`D_incident` (unseen)	43.0 (CoCycle)	50.4 (+17%)	49.5 (+15%)	67.8 (+58%)	101.2 (+135%)

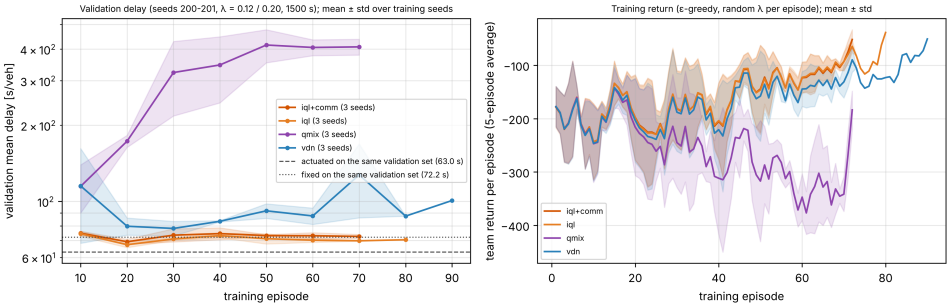


Figure 10.12: E12: validation delay during training (mean over training seeds).

IQL comes within 7–19% of the best handcrafted controller and never beats it; its gap on unseen patterns is similar to that on seen demand, so it generalises. Neighbour observations add nothing and hurt on the unseen hotspot. VDN and QMIX are much worse, with gridlock in some `D_high` runs and a large spread over training seeds (QMIX best validation delay 82–142 s). A learning rate of 10^{-4} stabilises VDN (45.0 / 114.5 / 53.2 s on med / high / tidal), but both still trail IQL by 8–22% and QMIX still fails at `D_high`. **Why.** About 70–90 one-hour episodes with a queue-based reward are not enough to discover better switching than a tuned rule, and value factorisation over 9 agents adds variance (credit assignment).

10.15 E13 – DMPC, STL constraints and epistemic guards

Table 10.6: E13: mean delay [s] / spillback events per hour (5 seeds). Overload = 0.25 veh/s per entry.

controller	overload, 0% loss	overload, 80% loss	`D_high`, 80% loss
CoMP	428.7 / 284	416.6 / 353	102.5 / 0
CoMP + epistemic guard	470.9 / 205	713.3 / 211	303.2 / 43
DMPC	400.8 / 232	403.4 / 193	106.6 / 3
DMPC + STL	397.5 / 126	400.6 / 166	106.2 / 0
DMPC + STL + epistemic	407.3 / 69	776.8 / 138	433.5 / 80

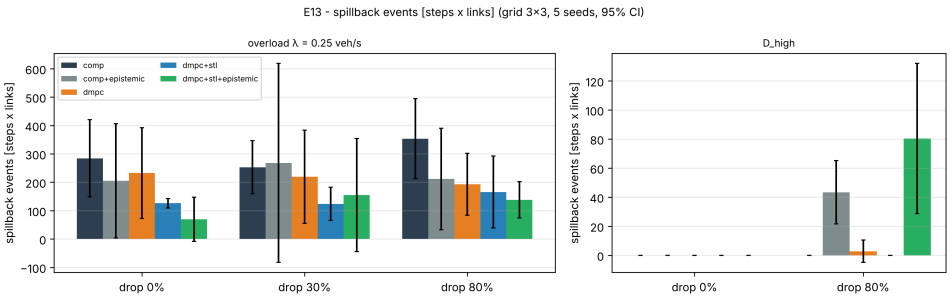


Figure 10.13: E13: spillback events per hour by controller and message loss.

DMPC beats CoMP by 3.2–6.5% under overload at every loss level (CIs exclude 0): when queues interact, predicting neighbours' departures helps. It ties CoMP at `D_high` without loss (−0.8%, n.s.) and is 4.0% worse at 80% loss. STL constraints cut spillback by 43–46% relative to DMPC at 0–30% loss (14% at 80%) at no delay cost; DMPC+STL is the fastest controller under overload (−7.3% vs CoMP). The epistemic guard gives the least spillback when messages arrive (69 events vs 284 for CoMP), but under heavy loss knowledge vanishes, the guard blocks most releases and delay rises by 87% (overload) to 323% (`D_high`). Under overload $\rho(\Phi) < 0$ for every controller (some link is always full); no phase was ever starved (maximum red 48 s vs $R_{\max} = 120$ s). With 5 seeds the minimum Wilcoxon p is 0.0625, so these conclusions rest on the paired CIs.

10.16 E14 – Incident and tidal scenarios

Incident (10 seeds; DMPC+STL 5). Recovery time is measured from the end of the incident until the network queue is within 10% of its pre-incident level: actuated 374 s, CoMP 367 s (−1.7%, n.s.), DMPC+STL 364 s, CoCycle 339 s (−9.2% [−22.4, +0.2]) with the lowest delay (−8.4%), CoMP+Cycle 577 s, and fixed-time 1079 s with 2 of 10 runs never recovering within the hour (Figure 10.14). **Tidal.** CoCycle has the lowest delay (41.7 s, −7.9% vs actuated); CoMP and DMPC+STL tie actuated, CoMP+Cycle loses 31%. Queues above $0.6C$ almost never occur in the tidal scenario, so the clearance metric does not discriminate there.

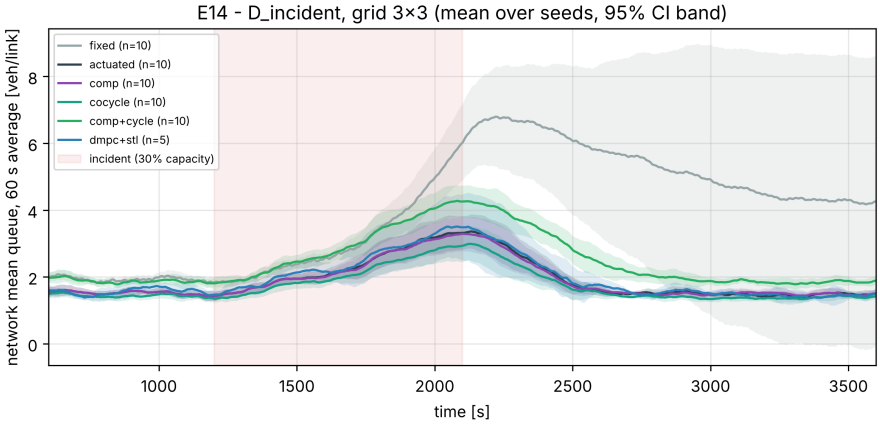


Figure 10.14: E14 (F09): network mean queue around the incident (shaded: incident window).

10.17 Time–space diagram and demonstration

Figure 10.15 shows vehicle trajectories and signal states along the middle eastbound arterial for fixed, actuated and CoCycle control (`D_med`, seed 0). Each line is a vehicle; horizontal segments

are waiting at a stop line, and the coloured bars show the signal state at each stop line. The diagram makes visible how each controller times the greens relative to arriving platoons; as the ablations show, the offset alignment produced by CoCycle does not by itself reduce delay. `results/demo/` contains side-by-side animations of the three controllers on identical vehicles.

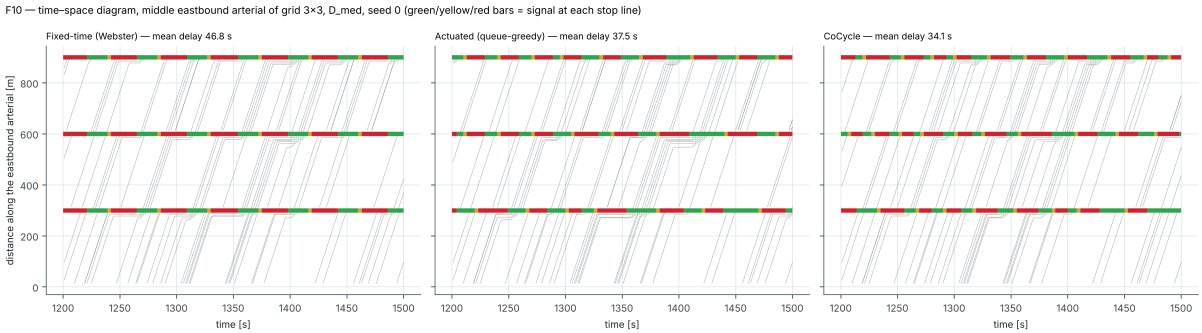


Figure 10.15: F10: time–space diagram of the middle eastbound arterial (grid3x3, `D_med`, 1200–1500 s).

10.18 SUMO validation and real-city scenarios

Grid co-simulation. The TrafficSync grid is converted to a SUMO network (one edge per link with a 60 m left-turn bay), the same vehicles are inserted, and the unchanged controllers drive SUMO's signals through the same message bus (5 seeds). Without the left-turn bay, a left-turner waiting for a gap blocks the single lane and delay at `D_med` was 161 s vs 47 s in the queue model. With it, fixed-time vs actuated (+7.2% for fixed at `D_med`) and CoMP = actuated carry over, but **CoCycle's advantage does not**: in SUMO it is 23.2% worse than actuated at `D_med` (Figure 10.16). At `D_high` the SUMO grid partly gridlocks and is not used. CoCycle's gain is therefore specific to the point-queue model.

Real cities. Six real-world scenarios with measured demand and the cities' own signal plans: Cologne with 3 and 8 signals (TAPASCologne [29]) and Ingolstadt with 7 and 21 signals (InTAS [30]), both from the RESCO benchmark [31], and Bologna Andrea Costa (7) and Pasubio (8, with buses) [32]. Every signal becomes an agent whose phases are the main green phases of the city plan; agents use lane detectors on their own approaches and exit lanes; CoMP and CoCycle exchange messages with road neighbours. Margins and green limits were tuned per city on SUMO seed 100; evaluation used seeds 0–2.

Table 10.7: Real cities in SUMO: mean delay [s], 3 seeds; best agent in bold.

city (signals)	city plan	actuated	max-pressure	CoMP	CoCycle	best vs plan
Cologne (3)	35.5	**22.8**	34.5[a]	42.4[a]	25.9	−36%
Cologne (8)	51.2	25.5	**24.2**	24.8	27.3	−53%
Ingolstadt (7)	124.7	59.3	**54.8**	64.3	82.8	−56%
Ingolstadt (21)	139.0	98.6	**83.9**	85.0	86.5	−40%
Bologna A. Costa (7)	222.1	269.0[b]	**205.8**	261.9	267.3[b]	−7%
Bologna Pasubio (8)	204.4	**129.5**	132.2	133.3	180.4	−37%

[a] one of three seeds jammed (very wide CI). [b] fewer completed trips than the city plan (4103 / 5169 vs 5988).

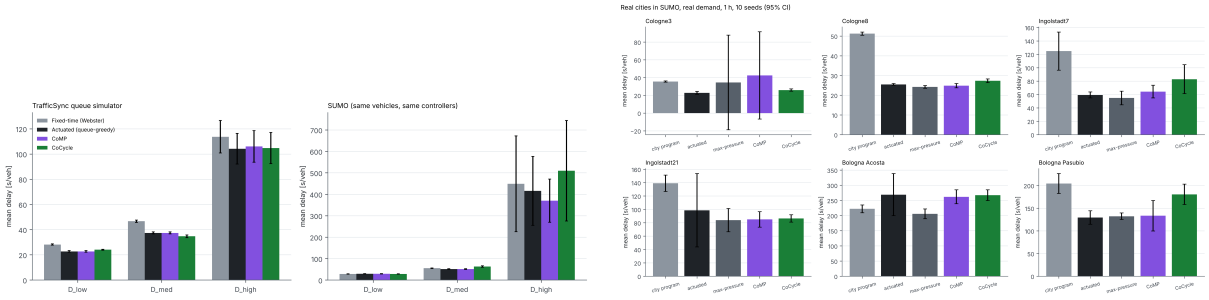


Figure 10.16: Left: the same vehicles and controllers in the queue simulator and in SUMO. Right: delay per controller in the six real cities.



Figure 10.17: Browser replay of Bologna Pasubio: all five controllers side by side on the same vehicles (`results/SUMO/web/`).

Reading. In all six scenarios the best agent design beats the city's plan, by 7–56%. Local max-pressure is best in four cities; CoMP roughly matches it and is unchanged at 100% message loss on Cologne; CoCycle beats the city plan in five cities but is not the best design anywhere. In Bologna Andrea Costa, actuated and CoCycle agents complete fewer trips than the city plan, so their delays understate their cost; there only max-pressure improves on the plan.

Chapter 11

Discussion and Critical Interpretation

11.1 When does coordination actually help?

Across all experiments, communication changed traffic outcomes in only three situations: (i) when local information was poor, neighbour reports improved max-pressure (CoMP -3.5% at `D_high` and -2.2% at `D_ramp` with 20% probes); (ii) under overload, where queues interact strongly, DMPC that predicts neighbours' departures beat CoMP by 3–7% and, with STL constraints, roughly halved spillback; and (iii) under faults, where the *choice* of aggregation rule (W-MSR vs averaging) decided whether a single faulty agent could add 19% delay. With perfect local detectors and moderate demand, the information that neighbours could share was either already implicit in the local queues or irrelevant to the switching decision.

11.2 Why CoCycle wins in the queue model

CoCycle beat actuated control by 6.7% (`D_med`) and 8.1% (`D_tidal`) over 30 seeds, a large and highly significant effect. The ablations, however, show the same gain with no consensus, no offsets, any communication graph and 100% message loss. What remains is the local structure: a Webster split plan with gap-out and a bounded extension window. A plausible explanation is that the plan prevents the queue-greedy oscillation of actuated control, in which a phase is held as long as its queue is marginally larger, and instead serves both approaches in proportion to their flow ratios; we did not isolate this mechanism. The gain disappeared in SUMO (+23% vs actuated), where queues form gradually along the link rather than at a point and the halting-vehicle count is a different queue measure. The honest conclusion is that CoCycle's advantage is a property of its local plan *in the point-queue model*, not evidence for coordination.

11.3 Why CoMP behaves like independent control

CoMP adds the downstream queue and a spillback veto to max-pressure. With 300 m links and demand below saturation, downstream links are rarely close to full, so $\tilde{q}_m$ is small compared with the tuned margin $\theta = 8$ and the switching decision is the same as without it; anticipation ($\kappa > 0$) did not help on tuning seeds; and the consensus weight γ_i is close to 1 on a homogeneous grid. On real city maps, where approaches have several lanes, CoMP is again close to local max-pressure, and with 100% loss it becomes identical to it — graceful degradation by construction.

11.4 Why noisy probe information degrades adaptive control

An adaptive controller is a feedback loop on the measured queue. With $\pi = 0.2$ the estimate is a scaled count of one or two probes, so it is zero for most real queues and jumps by five vehicles when a probe stops. The controller then switches on phantom queues and misses real ones, paying 4 s of lost time per switch. A fixed plan has no feedback and therefore no noise amplification. Only when the plan is badly wrong (`D_ramp`) does adaptation outweigh the noise. This is why information *quality* matters more than information *sharing*: sharing noisy estimates with neighbours does not remove the noise, while better coverage ($\pi \geq 0.5$) restores the benefit of adaptation.

11.5 Why connectivity does not improve performance

The theory is exact and was confirmed: consensus and gradient-tracking iterations scale with λ_2 (Spearman -0.99 and -0.95). But traffic performance depends on the agreed value only if the controller's performance depends on agreement, and the ablations show that it does not. Faster agreement on a quantity that does not affect delay cannot reduce delay. This illustrates a general lesson for multi-agent design: theoretical guarantees are about the coordination layer; whether they translate into system performance depends on the coupling between that layer and the objective.

11.6 Why more complex controllers can perform worse

CoMP+Cycle combined two mechanisms that both extend greens, and the combination was worse than either. DMPC with a pure quadratic objective was 3× worse than CoMP until the objective was changed to reflect delay and switching cost. VDN and QMIX, with more expressive training signals than IQL, were less stable. Each additional mechanism adds parameters, interactions and failure modes; without careful tuning and a matching objective, complexity does not translate into performance.

11.7 Overload and spillback

Near and beyond saturation (E4 $\lambda \geq 0.23$, E13) delays are dominated by queues that fill links and block upstream junctions. Here coordination mechanisms that act on space downstream — DMPC's predicted arrivals, STL safety constraints, the epistemic guard — reduce spillback substantially, and the epidemic analysis distinguishes local contagion (fixed-time under ramp and incident, $R > 1$) from global saturation (overload, $\hat{\beta} \approx 0$). The knowledge-based guard illustrates the safety–efficiency trade-off: it is the safest controller when communication works and the slowest when it does not.

11.8 Theory and practice

Where the assumptions of a theorem hold in our setting, the measured behaviour matched it closely: consensus stability bound and λ_2 rate law (E8), W-MSR invariance and the necessity of robustness (E9), exactness of gradient tracking and bias of constant-step DGD (E10), the spectral threshold separating spreading from non-spreading scenarios (E11). The weak links were between these verified components and traffic delay, not inside the components.

11.9 Hypotheses

Table 11.1: Verdicts on the hypotheses.

H1	**Not supported as a coordination effect.** Actuated beats fixed-time (8–49%) and CoCycle beats actuated at `D_med`/`D_tidal` ($-6.7\%/-8.1\%$), but the gain survives without communication and did not transfer to SUMO. Communication helped with probes (CoMP -3.5% at `D_high`) and under overload (DMPC).
H2	**Not supported.** No trend with network size; the gain is non-monotonic in demand (loss at light demand, gains at moderate and oversaturated demand, none at 0.20 veh/s).
H3	**Not supported for traffic performance** (delay 34.2–34.8 s on 44 graphs, Spearman +0.19); supported for consensus and optimiser convergence speed.
H4	**Supported, trivially**: CoMP and CoCycle keep their loss-free delay up to 100% loss; knowledge-based guards degrade sharply.
H5	**Supported with conditions**: W-MSR on static consensus over a $(2, 2)$-robust graph (+3 links) keeps CoMP within +0.6% (vs +18.8%); W-MSR with dynamic consensus fails.
H6	**Partly supported**: centrality correlates with spreading (+0.73), but prioritisation had no material effect.

Chapter 12

Limitations and Future Work

12.1 Limitations

- **Point-queue traffic model.** Queues form at a point, there is no car-following, lane changing or turning conflict, and left turns do not block through traffic. SUMO showed that at least one conclusion (CoCycle's advantage) depends on this simplification.
- **Two-phase signals** on the synthetic networks; protected left phases and pedestrian phases were not modelled. The real-city agents use the cities' own (multi-phase) plans but drop transitional phases.
- **Probe data are emulated**, not real; the noise and delay model is simple.
- **Demand** is synthetic on the grids; only the six real-city scenarios use measured demand, evaluated with 3 seeds.
- **Seeds**: DMPC experiments (E13, the DMPC part of E14) use 5 seeds instead of 10, so they rely on bootstrap CIs (minimum Wilcoxon $p = 0.0625$).
- **Epidemic analysis** uses a mean-field model and a threshold-based infection definition; congestion is not a contagion in a strict sense.
- **DMPC cost**: one QP per agent every 5 s; the 9-agent experiments took about 2 h on 2 cores.
- **MARL budget**: about 20 thousand gradient steps per policy and three training seeds.
- **Communication** is idealised apart from latency, loss, period and outages (no bandwidth limits or correlated losses); faulty agents are modelled only as faulty reporters.
- **No real-world deployment.**
- **Not implemented** (optional extensions): MAPPO, emergency-vehicle preemption, four-phase signals.

12.2 Future work

- Run the full experimental design in SUMO (multi-lane approaches, protected phases) to see which conclusions are model-independent, and isolate the mechanism behind CoCycle's local gain.
- Use real probe and loop-detector data, and calibrate demand for larger city networks.

- Four-phase signals and emergency-vehicle priority (leader–follower with knowledge acknowledgement $K_i K_j$).
- MARL with larger budgets, curriculum on demand, MAPPO and graph neural network policies; reward shaping toward delay instead of queue.
- Coordination focused on the regime where it mattered here: overload and spillback (DMPC with STL constraints, perimeter control).
- Resilient dynamic consensus (e.g. W-MSR with periodic resets) so that resilience and tracking can be combined.
- Communication models with bandwidth limits and correlated outages; real-time deployment on hardware-in-the-loop controllers.

Chapter 13

Conclusion

TrafficSync set out to answer when sharing traffic information between signalised intersections improves network performance. We modelled each intersection as an autonomous agent with local sensing, local actuation and messages over an imperfect communication graph, built a deterministic and validated traffic simulator in which every controller sees identical vehicles, and implemented a family of controllers that apply the central ideas of the course: graph Laplacians and algebraic connectivity, average and dynamic consensus, formation control for green waves, resilient W-MSR consensus with (r, s)-robustness, distributed optimisation by ADMM and gradient tracking, distributed MPC with Signal Temporal Logic constraints, epistemic knowledge guards, spectral epidemic analysis and multi-agent reinforcement learning.

The experiments, paired over common seeds with bootstrap intervals and corrected tests, give a clear and partly negative picture. Adaptive local control beats fixed-time plans by 8–49% with good detectors, and in six real-city SUMO scenarios our agents beat the cities' own plans by 7–56%. The best coordinated controller, CoCycle, reduces delay by a further 6.7–8.1% in the queue model, but ablations, topology sweeps and message-loss experiments show that this gain is produced by its local plan structure, and it does not transfer to SUMO. Information quality dominates information sharing: with 20% probe vehicles adaptive controllers are worse than fixed-time. Communication made a measurable difference when local information was noisy, under overload, where prediction and safety constraints reduced spillback, and under faults, where the aggregation rule decided whether one agent could corrupt the network. The distributed-systems theory was confirmed wherever its assumptions held.

The main lesson is that coordination layers can work exactly as theory predicts and still not improve the system objective, unless the objective actually depends on what the agents agree on. TrafficSync's contribution is a reproducible platform and a rigorous experimental study that separates these two questions, together with the tools — SUMO co-simulation, real-city scenarios and side-by-side replays — to test future coordination schemes against the same standard.

Appendix A

Team, Reproducibility and Parameters

A.1 Individual contributions

Member	Contribution
Abhijeet (ai24btech11001)	**Network and communication layer.** Road and communication graphs, network generators, Laplacian and spectral tools, (r, s)-robustness and graph augmentation, message format and communication bus (latency, loss, outages). Experiments E6 (communication topology), E7 (communication failures), E8 (consensus validation), E9 (W-MSR resilience) and E10 (ADMM, DGD, gradient tracking).
Charan (ai24btech11022)	**Coordinated controllers and formal methods.** CoMP, CoCycle (consensus cycle and formation-control offsets), CoMP+Cycle, W-MSR integration into the controllers, DMPC with STL constraints and the epistemic guard, STL monitor and Kripke models. Experiments E2 (main comparison), the ablations and E13 (DMPC, STL, epistemic guard).
Sumanth (ai24btech11027)	**Analysis, visualisation and SUMO.** Statistical methods (bootstrap CIs, Wilcoxon, Holm), plots and summary figures, animations and dashboard, epidemic analysis (E11), SUMO co-simulation of the grid, real-city agents for six cities, OpenStreetMap import, per-city tuning and the interactive browser replays.
Shreyansh (ai24btech11032)	**Traffic simulator and experiment engine.** Links, vehicles, intersections and the signal state machine, demand scenarios, probe-vehicle information model, metrics logger, configuration system, episode runner and experiment engine, regression tests. Experiments E0 (simulator validation), E1 (baselines), E3 (network size), E4 (demand sweep) and E5 (information quality).
Tanush (ai24btech11034)	**Local controllers and learning.** Fixed-time (Webster), actuated and local max-pressure control, the shared switching rule, parameter tuning, multi-agent reinforcement learning (environment, IQL, IQL+comm, VDN, QMIX, training and evaluation). Experiments E12 (MARL) and E14 (incident and tidal stress tests).

Parts of the code, experiment scripts and text were produced with the assistance of an AI assistant (Claude, Anthropic); all reported numbers were produced by running the code in the repository.

A.2 Reproducibility

```
pip install -r requirements.txt # numpy, scipy, matplotlib, pyyaml, cvxpy, torch
pip install -r requirements-sumo.txt # optional: eclipse-sumo, traci, sumolib
python -m pytest -q # 166 tests incl. regression and SUMO
python experiments/tune.py --workers 2 # tuning on seeds 100-102
python experiments/core_experiments.py --workers 2 # E0-E8 and ablations
python experiments/e9_resilience.py --seeds 10 # E9 (similarly e10..e14, e12_marl)
python experiments/sumo_tune.py && python experiments/sumo_experiments.py --map-seeds 3
python experiments/make_web_viewer.py # browser replays -> results/SUMO/web/index.html
./run_all.sh 2 # everything
```

Every result in this report is stored in `results/<experiment>/` (`runs.csv`, `summary.md`, figures).

A.3 Software

Python 3.10+, NumPy, SciPy, Matplotlib, PyYAML, CVXPY with OSQP and CLARABEL, PyTorch (MARL), pytest; Eclipse SUMO 1.2x with TraCI and sumolib (co-simulation); Leaflet (browser replays).

A.4 Default parameters

Table A.1: Main simulation and controller parameters.

Simulation	$\Delta t = 1$ s, 3600 s episodes, 600 s warm-up, decision interval 5 s, turning ratios 0.70/0.15/0.15
Signals	$g_{\min} = 5$ s, $g_{\max} = 40$ s, yellow 3 s, all-red 1 s, start-up lost time 2 s
Links	300 m (grid) / 400 m (corridor), $v_f = 13.9$ m/s, jam spacing 7.5 m, $s_l = 0.5$ veh/s/lane
Probes	$\pi = 0.2$, report period 5 s, speed noise 1 m/s, position noise 5 m, delay 2 s, EMA 0.3
Communication	period 5 s, latency 1 step, drop 0, topology enforced
Fixed-time	Webster, $L_{tot} = 12$ s, $C \in [30, 120]$ s
Actuated / max-pressure	$\theta = 16$ / 8, gap-out
CoMP	$\theta = 8$, $\kappa = 0$, anticipation 15 s, $\delta_{room} = 2$, max info age 30 s, $\gamma \in [0.5, 1.5]$, $\beta_c = 1$, $K_c = 5$, dynamic consensus
CoCycle	Webster rule, $C \in [40, 100]$ s, quantum 4 s, hysteresis 3 s, $K_c = 5$, offsets on, max extension 40 s
W-MSR	$F = 1$, static consensus
DMPC	$H = 12 \times 5$ s, $w_{\text{lin}} = 1$, $w_q = 0.05$, $w_{\text{sw}} = 30$ (L1), decision look-ahead 3 steps, margin 1.5, sequential consistency
STL	$\delta = 2$, clearance $0.6C \to 0.2C$ within 180 s, $R_{\max} = 120$ s
Epistemic guard	$\delta_0 = 2$, $\lambda^{\max} = s_m$
MARL	MLP 2×64, replay 10^5, batch 64, $\gamma = 0.99$, target update 200, lr 10^{-3} (IQL) / 5×10^{-4} (VDN, QMIX)

A.5 Consensus on the 3-node path

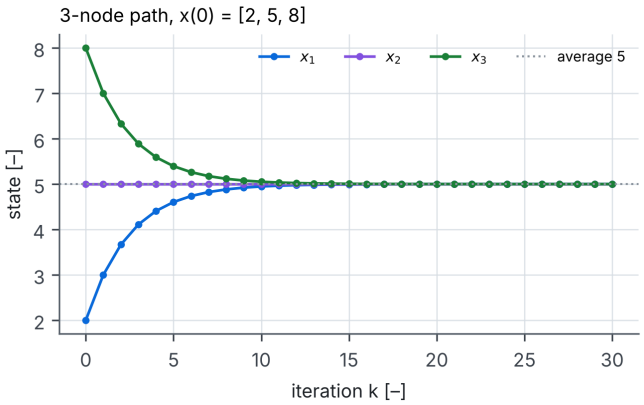


Figure A.1: Laplacian consensus on a 3-node path from $x(0) = [2, 5, 8]$.

A.6 Demonstration still

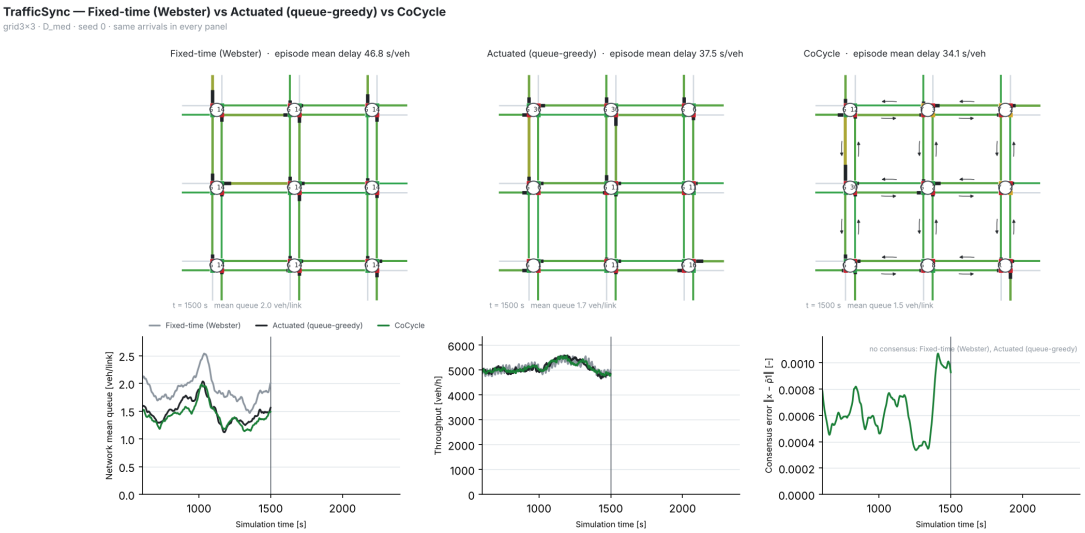


Figure A.2: Side-by-side animation (fixed, actuated, CoCycle) on grid3x3, `D_med`, $t = 1500$ s: links coloured by occupancy, signal heads and messages (`results/demo/`).

Bibliography

[1] F. V. Webster. *Traffic Signal Settings*. Road Research Technical Paper No. 39. HMSO, London, 1958.

[2] Pravin Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C*, 36:177–195, 2013.

[3] Pablo Alvarez Lopez et al. Microscopic traffic simulation using SUMO. In *21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, pages 2575–2582, 2018.

[4] Minghui Zhu and Sonia Martínez. Discrete-time dynamic average consensus. *Automatica*, 46(2):322–329, 2010.

[5] Heath J. LeBlanc, Haotian Zhang, Xenofon Koutsoukos, and Shreyas Sundaram. Resilient asymptotic consensus in robust networks. *IEEE Journal on Selected Areas in Communications*, 31(4):766–781, 2013.

[6] Angelia Nedić and Asuman Ozdaglar. Distributed subgradient methods for multi-agent optimization. *IEEE Transactions on Automatic Control*, 54(1):48–61, 2009.

[7] Guannan Qu and Na Li. Harnessing smoothness to accelerate distributed optimization. *IEEE Transactions on Control of Network Systems*, 5(3):1245–1260, 2018.

[8] Wei Shi, Qing Ling, Gang Wu, and Wotao Yin. Extra: An exact first-order algorithm for decentralized consensus optimization. *SIAM Journal on Optimization*, 25(2):944–966, 2015.

[9] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, and Jonathan Eckstein. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends in Machine Learning*, 3(1):1–122, 2011.

[10] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.

[11] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proc. AAAI*, pages 2094–2100, 2016.

[12] Ming Tan. Multi-agent reinforcement learning: Independent vs. cooperative agents. In *Proc. ICML*, pages 330–337, 1993.

[13] Peter Sunehag et al. Value-decomposition networks for cooperative multi-agent learning based on team reward. In *Proc. AAMAS*, pages 2085–2087, 2018.

[14] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *Proc. ICML*, pages 4295–4304, 2018.

[15] Eduardo Camponogara, Dong Jia, Bruce H. Krogh, and Sarosh Talukdar. Distributed model predictive control. *IEEE Control Systems Magazine*, 22(1):44–52, 2002.

[16] Steven Diamond and Stephen Boyd. CVXPY: A python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[17] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[18] Oded Maler and Dejan Nickovic. Monitoring temporal properties of continuous signals. In *Proc. FORMATS/FTRTFT*, volume 3253 of *LNCS*, pages 152–166, 2004.

[19] Alexandre Donzé and Oded Maler. Robust satisfaction of temporal logic over real-valued signals. In *Proc. FORMATS*, volume 6246 of *LNCS*, pages 92–106, 2010.

[20] Ronald Fagin, Joseph Y. Halpern, Yoram Moses, and Moshe Y. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.

[21] Yang Wang, Deepayan Chakrabarti, Chenxi Wang, and Christos Faloutsos. Epidemic spreading in real networks: An eigenvalue viewpoint. In *Proc. SRDS*, pages 25–34, 2003.

[22] Romualdo Pastor-Satorras, Claudio Castellano, Piet Van Mieghem, and Alessandro Vespignani. Epidemic processes in complex networks. *Reviews of Modern Physics*, 87(3):925–979, 2015.

[23] Carlos F. Daganzo. The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory. *Transportation Research Part B*, 28(4):269–287, 1994.

[24] J. K. Terry et al. PettingZoo: Gym for multi-agent reinforcement learning. NeurIPS 2021, 2021.

[25] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1994.

[26] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

[27] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2): 65–70, 1979.

[28] Raj Jain, Dah-Ming Chiu, and William R. Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. In *DEC Research Report TR-301*, 1984.

[29] Sandesh Uppoor, Oscar Trullols-Cruces, Marco Fiore, and Jose M. Barcelo-Ordinas. Generation and analysis of a large-scale urban vehicular mobility dataset. In *IEEE Transactions on Mobile Computing*, volume 13, pages 1061–1075, 2014.

[30] Silas C. Lobo, Stefan Neumeier, Evelio M. G. Fernandez, and Christian Facchi. InTAS – the Ingolstadt traffic scenario for SUMO. In *SUMO User Conference*, 2020.

[31] James Ault and Guni Sharon. Reinforcement learning benchmarks for traffic signal control. In *NeurIPS Datasets and Benchmarks Track*, 2021.

[32] Laura Bieker, Daniel Krajzewicz, AntonioPio Morra, Carlo Michelacci, and Fabio Cartolano. Traffic simulation for all: a real world traffic scenario from the city of Bologna. In *SUMO 2014 – Modeling Mobility with Open Data*, Lecture Notes in Mobility, pages 47–60. Springer, 2015.